گزارش نهایی پروژه‌ی تحقیق در عملیات 2

دانشکده مهندسی صنایع - ارائه شده توسط دکتر رادمان

بهار 1404

نام و نام خانوادگی اعضا	شماره‌ی دانشجویی اعضا
پارسا کشمیری	401104373
علیرضا فرجی	401104327
محمدعرفان کلماتیان	401104384
آمیار سی و سه مرده	401104192

فهرست گزارش

بخش دوم) جواب بهینه مدل **3**
بخش سوم) ذخیره و مصورسازی جواب **10**
بخش چهارم) ریلکس کردن محدودیت اسکله سوم **15**
بخش ششم) تحلیل حساسیت **22**
زمان بین تخلیه دو کشتی در اسکله دو) **22**
زمان تخلیه کشتی شماره هفت روی اسکله شماره سه) **28**
ظرفیت اسکله شماره دو) **34**
ظرفیت اسکله شماره سه) **41**
زمان رسیدن کشتی شماره شش) **46**
گزارش بخش :Gurobipy **51**

بخش دوم) جواب بهینه مدل

در این بخش ابتدا کتابخانه‌های مربوطه را وارد کردیم:

```
import pyomo.environ as pyo
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import ast
from pyomo.environ import value, TerminationCondition
import seaborn as sns
import math
```

پس از تعریف کردن مدل، ابتدا مجوعه‌ها و اندیس‌ها را تعریف کردیم:

```
model = pyo.ConcreteModel()
n=8
m=4
#ship sets
model.i = pyo.RangeSet(1,n)
model.q= pyo.RangeSet(1,n)
model.iq = pyo.Set(dimen=2, initialize=lambda m: [(i, q) for i in model.i for q
in model.i if i != q])
#doc set
model.j= pyo.RangeSet(1,m)
# objective set
model.k = pyo.RangeSet(1,3)
model.bin = pyo.RangeSet(1, 2)
big_M = 80
```

پس از این مرحله نیاز به خوانش پارامترها از فایل اکسل داشتیم:

```
Containers= pd.read_excel('Parameters.xlsx', sheet_name='Containers',
index_col=0)
Anchorage = pd.read_excel('Parameters.xlsx', sheet_name='Anchorage',
index_col=0)
Unload_Time = pd.read_excel('Parameters.xlsx', sheet_name='Unload Time',
index_col=0)
Capacity = pd.read_excel('Parameters.xlsx', sheet_name='Capacity', index_col=0)
Length = pd.read_excel('Parameters.xlsx', sheet_name='Length', index_col=0)
Max_length = pd.read_excel('Parameters.xlsx', sheet_name='Max Length',
index_col=0)
```

```
Lag = pd.read_excel('Parameters.xlsx', sheet_name='Lag', index_col=0)
Priority = pd.read_excel('Parameters.xlsx', sheet_name='Priority', index_col=0)
Arrival_Time= pd.read_excel('Parameters.xlsx', sheet_name='Arrival Time',
index_col=0)
Cost_for_Extra_Capacity= pd.read_excel('Parameters.xlsx', sheet_name='Cost for
Extra Capacity', index_col=0)
Max_Extra_Allowable_Capacity = pd.read_excel('Parameters.xlsx', sheet_name='Max
Extra Allowable Capacity', index_col=0)
df = pd.read_excel("Parameters.xlsx", sheet_name="Unload Time")

# Convert the 'Ship_Berth' string to a tuple of integers
df['Ship_Berth'] = df['Ship_Berth'].apply(lambda x: tuple(map(int,
ast.literal_eval(x))))

# Now create the dict
unload_dict = dict(zip(df['Ship_Berth'], df['Unload_Time']))
```

پس از خوانش پارامترها می‌توانستیم پارامترهای مدل را تعریف کنیم:

```
model.C = pyo.Param(model.j, initialize= Capacity.loc[:,'Capacity'].to_dict(),
mutable= True)
model.CA= pyo.Param(model.i, initialize=
Containers.loc[:,'Containers'].to_dict(), mutable= True)
model.L= pyo.Param(model.j, initialize=
Max_length.loc[:,'Max_Length'].to_dict(), mutable= True)
model.LS= pyo.Param(model.i, initialize= Length.loc[:,'Length'].to_dict(),
mutable= True)
model.P= pyo.Param(model.i, initialize= Priority.loc[:,'Priority'].to_dict(),
mutable= True)
model.O = pyo.Param(model.j, initialize= Lag.loc[:,'Lag'].to_dict(), mutable=
True)
model.Un = pyo.Param(model.i, model.j, initialize= unload_dict, mutable= True)
model.a = pyo.Param(model.i, initialize=
Arrival_Time.loc[:,'Arrival_Time'].to_dict(), mutable= True)
model.CIC = pyo.Param(model.j, initialize= Cost_for_Extra_Capacity.loc[:,'Cost
for Extra Capacity'].to_dict(), mutable=True)
model.ext = pyo.Param(model.j, initialize=
Max_Extra_Allowable_Capacity.loc[:,'Max Extra Allowable Capacity'].to_dict(),
mutable= True)
```

به طریق مشابه متغیرهای تصمیم مدل را تعریف می‌کنیم فقط در اینجا به دلیل اقتضائات پیش آمده ناچار به ایجاد برخی تغییرات در مدل اصلی شدیم، زمان معطلی هر کشتی را از W_{ij} به

W_i تغییر دادیم زیرا در غیر این صورت در محدودیت تعادلی مربوط به مدت زمان تخلیه کشتی، اگر کشتی به اسکله مربوطه اختصاص نیافته بود عبارات سمت چپ تساوی صفر می‌شدند در حالی که سمت راست تساوی یک پارامتر غیر صفر بود.
همچنین متغیر z_{iq} را به علت همپوشانی با y_{iqj} حذف کردیم. شایان ذکر است که چون این متغیر از مقادیر مسئله بدست می‌آمد نیازی به تعریف آن نبود.
در نهایت متغیر s_i را به عنوان زمان شروع خدمت‌دهی به کشتی i ام تعریف کردیم. همچنین یک اندیس مربوط به هر کشتی نیز به u اضافه شده است.

```
model.X = pyo.Var(model.i, model.j, domain=pyo.Binary)
model.W = pyo.Var(model.i, domain= pyo.NonNegativeReals)
model.d = pyo.Var(model.i, model.j, domain= pyo.NonNegativeReals)
model.IC = pyo.Var(model.j, domain=pyo.NonNegativeIntegers)
model.y = pyo.Var(model.iq, model.j, domain= pyo.Binary)
model.u = pyo.Var(model.i, model.k, domain= pyo.NonNegativeReals)
model.f = pyo.Var(model.i, model.bin, domain=pyo.Binary)
model.s = pyo.Var(model.i, domain= pyo.NonNegativeReals)
```

پس از طی این آماده‌سازی‌ها می‌توانیم تابع هدف و پس از آن محدودیت‌ها را تعریف کنیم، فقط لازم به ذکر است که متغیر u که برای خطی‌سازی W تعریف کرده بودیم بدین صورت با W ارتباط پیدا می‌کند:

$$W_i = \sum_k u_{ik}$$

همچنین تعدادی محدودیت هم بعدا به همین دلیل اضافه می‌شوند.

```
def objective_function_expression(model):
  return 25*pyo.quicksum(model.u[i,1] for i in model.i) + 10 *
pyo.quicksum(model.u[i,2] for i in model.i) + 5*pyo.quicksum(model.u[i,3] for i
in model.i)+ 5* pyo.quicksum(model.Un[i,j] * model.X[i,j] for i in model.i for
j in model.j) + pyo.quicksum(model.CIC[j]* model.IC[j] for j in model.j)
model.obj = pyo.Objective(rule= objective_function_expression, sense=
pyo.minimize)
```

در رابطه با محدودیت‌ها لازم به ذکر است که این تغییرات را اعمال کردیم:

- محدودیت تعادلی زیر بین متغیر شروع خدمت‌دهی و زمان رسیدن کشتی و میزان صبر کردن آن تعریف کردیم:

$$W_i = s_i - a_i$$

- محدودیت شروع نکردن تخلیه بار کشتی بعدی قبل از تخلیه کامل کشتی قبل را نیز به این صورت اصلاح کردیم:

$$s_i + Un_{ij} + O_j \leq s_q + M(1 - y_{iqj}),\ \forall i, q, j, i \neq q$$

- در ادامه به منظور اینکه در حالات خاص به مشکل غیر موجه بودن بخورد نکنیم محدودیت مربوط به اولویت را به این صورت اصلاح کردیم، همچنین دو محدودیت بعدی را به منظور حفظ فلسفه محدودیت اول تعریف کردیم:

$$y_{iqj} + y_{qij} \leq 1,\ \forall i, q, i \neq q$$

$$y_{iqj} + y_{qij} \geq x_{ij} + x_{qj} - 1$$

$$y_{iqj} + y_{qij} \leq x_{ij}$$

- در مدل‌سازی اولیه زمان انتظار کشتی اول صفر نمی‌شد به همین علت دو محدودیت زیر (محدودیت اول برای اسکله سوم و دوم برای اسکله‌های دیگر) را به مدل‌سازی اضافه کردیم:

$$W_i \leq 80(2 - \sum_q y_{iq3}), \forall i, i \neq q$$

$$W_i \leq 80(1 - y_{iqj})$$

- در آخر محدودیت‌های مربوط به خطی‌سازی تابع هدف که ذکر کردیم را می‌آوریم:

$$10f_1 \leq u_1 \leq 10$$

$$30f_2 \leq u_2 \leq 30f_1$$

$$0 \leq u_3 \leq 30f_2$$

$$f_1 \geq f_2$$

```
def one_pier_to_ship(model, i):
  return pyo.quicksum(model.X[i,j] for j in model.j) ==1
model.c1= pyo.Constraint(model.i, rule=one_pier_to_ship)

def number_ships_to_pier(model, j):
  if j==3:
    return pyo.quicksum(model.X[i,j] for i in model.i) ==3
  else:
```

```
    return pyo.quicksum(model.X[i,j] for i in model.i) <=2
model.c2 = pyo.Constraint(model.j, rule= number_ships_to_pier)

def number_container(model, j):
  return pyo.quicksum(model.X[i,j] * model.CA[i]  for i in model.i) <=
model.C[j] + model.IC[j]
model.c3 = pyo.Constraint(model.j, rule=number_container)

def capacity_ceiling(model, j):
  return model.IC[j]<= model.ext[j]
model.c4 =pyo.Constraint(model.j, rule= capacity_ceiling)

def ship_length (model, i , j):
  return model.LS[i] * model.X[i,j] <= model.L[j]
model.c5 = pyo.Constraint(model.i, model.j, rule= ship_length)

def max_wait(model, i):
  return model.W[i] <= 80*(1-model.P[i])
model.c6 = pyo.Constraint(model.i, rule=max_wait)

def equ_cons_x_and_w(model, i):
  return model.W[i] == model.s[i] - model.a[i]
model.c7 = pyo.Constraint(model.i, rule=equ_cons_x_and_w)

def not_unloading(model, i, q, j):
    if i == q:
        return pyo.Constraint.Skip
    return  model.s[i] + model.Un[i,j] + model.O[j] <= model.s[q] + big_M *
(1-model.y[i, q, j])
model.c8 = pyo.Constraint(model.iq, model.j,  rule=not_unloading)

def equ_cons_y (model, i,q,j):
  if i==q:
    return pyo.Constraint.Skip
  return model.y[i,q,j]+ model.y[q,i,j]<=1
model.c9 = pyo.Constraint(model.iq, model.j, rule=equ_cons_y)


def y_one(model,i,q,j):
    if i==q:
        return pyo.Constraint.Skip
    return model.y[i,q,j] + model.y[q,i,j] >= model.X[i,j] + model.X[q,j] - 1
model.c10 = pyo.Constraint(model.iq, model.j, rule=y_one)
```

```
def y_two(model,i,q,j):
    if i==q:
        return pyo.Constraint.Skip
    return model.y[i,q,j] + model.y[q,i,j] <= model.X[i,j]
model.c11 = pyo.Constraint(model.iq, model.j, rule=y_two)


# محدودیت صفر شدن زمان انتظار اگر کشتی تنها باشد
def wait_zero_if_alone(model, i,q, j):
    if i==q:
        return pyo.Constraint.Skip
    if j==3 :
        y = pyo.quicksum(model.y[i,q,j] for q in model.q if q!= i)
        return model.W[i] <= 80 * (2- y)
    return model.W[i] <= 80 * (1 - model.y[i,q,j])
model.c13 = pyo.Constraint(model.iq, model.j, rule=wait_zero_if_alone)

def equ_cons_w_and_u (model, i):
  return model.W[i] == pyo.quicksum(model.u[i,k] for k in model.k)
model.c14 = pyo.Constraint(model.i, rule=equ_cons_w_and_u)

def equ_cons_f_and_u(model, i, bin):
  if bin == 1:
    return 10*model.f[i, bin]<= model.u[i, bin]
  elif bin == 2:
    return 30*model.f[i, bin]<= model.u[i, bin]
model.c15 = pyo.Constraint(model.i, model.bin, rule= equ_cons_f_and_u)

def equ_u(model, i, k):
  if k == 1:
    return 10 >= model.u[i, k]
  elif k == 2:
    return 30*model.f[i,1] >= model.u[i, k]
  elif k == 3:
    return 30*model.f[i,2] >= model.u[i, k]
model.c16 = pyo.Constraint(model.i, model.k, rule= equ_u)

def ff(model,i):
    return model.f[i,1] >= model.f[i,2]
model.c17 = pyo.Constraint(model.i, rule=ff)
```

پس از تعریف solver قطعه کد بعد از آن را به منظور نمایش خروجی به کار می‌بریم:

```
solver= pyo.SolverFactory('glpk')
```

```
result = solver.solve(model, tee=True)
result.write()
```

```
if result.solver.termination_condition != TerminationCondition.infeasible:
    for i, j in model.X.index_set():
        if model.X[i, j].value > 0:
            print(f"X[{i}, {j}] = {model.X[i,j].value}")
    for i in model.W.index_set():
        if model.W[i].value >= 0:
            print(f"W[{i}] = {model.W[i].value}")
    for i in model.s.index_set():
        if model.s[i].value >= 0:
            print(f"s[{i}] = {model.s[i].value}")

    for i,q, j in model.y.index_set():
        if model.y[i,q,j].value == 1:
            print(f"y[{i},{q}, {j}] = {model.y[i,q,j].value}")

    for i,j in model.u.index_set():
        if model.u[i,j].value > 0:
            print(f"u[{i},{j}] = {model.u[i,j].value}")


    obj_val = value(model.obj)
    print(f"Objective value: {obj_val}")
```

نتایج به شرح زیر خواهند بود:

```
X[1, 2] = 1.0
X[2, 4] = 1.0
X[3, 3] = 1.0
X[4, 1] = 1.0
X[5, 3] = 1.0
X[6, 2] = 1.0
X[7, 3] = 1.0
X[8, 4] = 1.0
W[1] = 0.0
W[2] = 0.0
W[3] = 19.2670572909898
W[4] = 0.0
```

```
W[5] = 39.3788913469807
W[6] = 20.6958212309185
W[7] = 0.0
W[8] = 8.24083515240811
s[1] = 4.76552698703798
s[2] = 1.00459250301696
s[3] = 22.9200559414305
s[4] = 6.05417812493725
s[5] = 41.2480808268544
s[6] = 22.813811450173
s[7] = 4.62295182452406
s[8] = 14.8420135358433
y[1,6, 2] = 1.0
y[2,8, 4] = 1.0
y[3,5, 3] = 1.0
y[7,3, 3] = 1.0
y[7,5, 3] = 1.0
u[3,1] = 10.0
u[3,2] = 9.26705729098979
u[5,1] = 10.0
u[5,2] = 29.3788913469807
u[6,1] = 10.0
u[6,2] = 10.6958212309185
u[8,1] = 8.24083515240811
Objective value: 1803.2706014701052
```

بخش سوم) ذخیره و مصورسازی جواب

ابتدا با استفاده از این کد جواب مدل و مقادیر متغیرها را ذخیره می‌کنیم.

```
if result.solver.termination_condition != TerminationCondition.infeasible:
    # Export X[i, j]
    X_data = [(i, j, model.X[i, j].value) for (i, j) in model.X if model.X[i,
j].value is not None and model.X[i, j].value > 0]
    df_X = pd.DataFrame(X_data, columns=['Ship', 'Dock', 'Assigned'])

    # Export W[i]
    W_data = [(i, model.W[i].value) for i in model.W if model.W[i].value is not
```

```
None]
    df_W = pd.DataFrame(W_data, columns=['Ship', 'Wait_Time'])

    # Export s[i]
    s_data = [(i, model.s[i].value) for i in model.s if model.s[i].value is not
None]
    df_s = pd.DataFrame(s_data, columns=['Ship', 'Start_Time'])

    # Export y[i, q, j]
    y_data = [(i, q, j, model.y[i, q, j].value) for (i, q, j) in model.y if
model.y[i, q, j].value == 1]
    df_y = pd.DataFrame(y_data, columns=['Ship1', 'Ship2', 'Dock', 'y_value'])

    # Export u[i, k]
    u_data = [(i, k, model.u[i, k].value) for (i, k) in model.u if model.u[i,
k].value is not None and model.u[i, k].value > 0]
    df_u = pd.DataFrame(u_data, columns=['Ship', 'Objective_K', 'u_value'])

    # Export IC[j]
    IC_data = [(j, model.IC[j].value) for j in model.IC ]
    df_IC = pd.DataFrame(IC_data, columns=['Dock', 'Extra_Capacity_Used'])

    #Exort Un[i,j]

    # Objective value
    obj_val = value(model.obj)

    df_orginal_solution = pd.concat([df_X, df_W], axis=1)
    # Save all to Excel
    with pd.ExcelWriter("pyomo_results.xlsx") as writer:
        df_X.to_excel(writer, sheet_name="X", index=False)
        df_W.to_excel(writer, sheet_name="W", index=False)
        df_s.to_excel(writer, sheet_name="s", index=False)
        df_y.to_excel(writer, sheet_name="y", index=False)
        df_u.to_excel(writer, sheet_name="u", index=False)
        df_IC.to_excel(writer, sheet_name="IC", index=False)

    print("Objective Value:", obj_val)
    print("Results saved to pyomo_results.xlsx")
```

در ادامه با استفاده از قطعه کد زیر اولویت بندی کرده و پس از آن به صورت گرافیکی تخصیص را مشاهده می کنیم:

```
priority_data = [(i, float(value(model.P[i]))) for i in model.P]
df_priority = pd.DataFrame(priority_data, columns=['Ship', 'Priority_Value'])

# Define priority levels
def classify_priority(p):
    if p > 0.9:
        return 'High'
    elif p > 0.4:
        return 'Medium'
    else:
        return 'Low'

df_priority['Priority'] =
df_priority['Priority_Value'].apply(classify_priority)
# Merge everything
df_schedule = df_X.merge(df_s, on="Ship").merge(df_Un,
on="Ship").merge(df_priority, on="Ship")
df_schedule['End_Time'] = df_schedule['Start_Time'] +
df_schedule['Unload_Duration']

# Color mapping by priority
priority_colors = {
    'High': '#ff6b6b',     # Red
    'Medium': '#feca57',   # Orange
    'Low': '#1dd1a1'       # Green
}

df_schedule['Color'] = df_schedule['Priority'].map(priority_colors)
```

پس از طبقه‌بندی براساس اولویت (بالا:قرمز،متوسط:نارنجی،پایین:سبز)، با استفاده از قطعه کد زیر نمودار گانت تخلیه بار کشتی‌های مختلف را رسم می‌کنیم:

```
# Make sure dock is numeric if it isn't already
df_schedule['Dock'] = df_schedule['Dock'].astype(int)

# Sort by Dock (numerically)
df_schedule = df_schedule.sort_values(by='Dock', ascending=False)

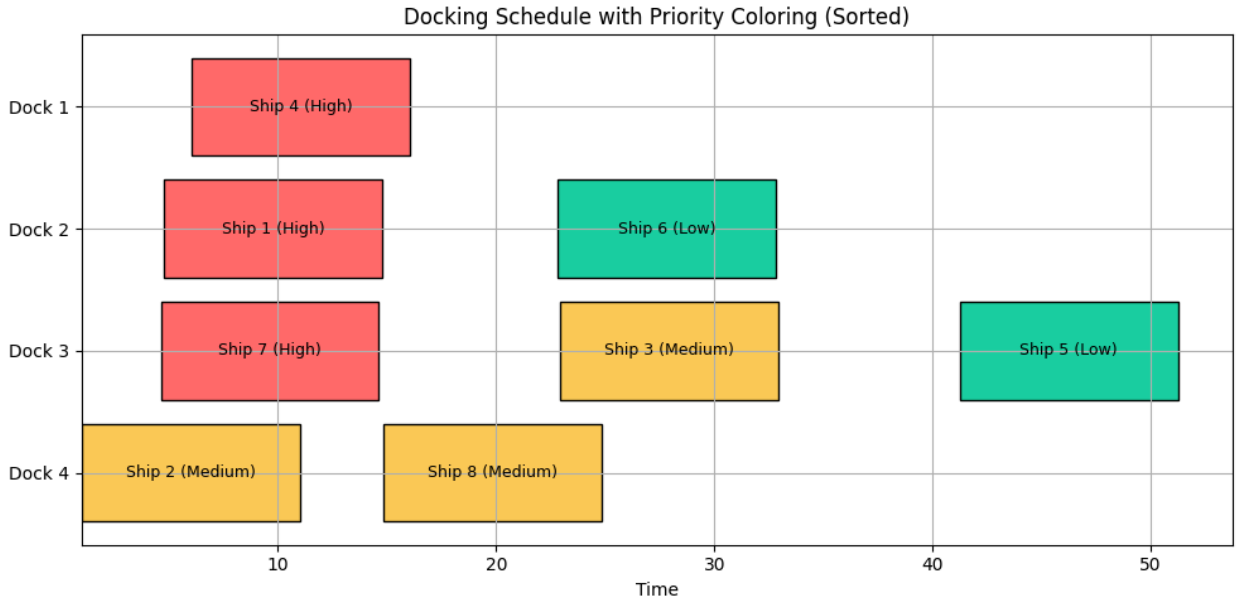
# Create an ordered list of dock labels
dock_labels = sorted(df_schedule['Dock'].unique())
```

```
dock_labels = [f'Dock {d}' for d in dock_labels]

# === Gantt Chart with sorted docks ===
fig, ax = plt.subplots(figsize=(10, 5))
for _, row in df_schedule.iterrows():
    ax.barh(y=f'Dock {row.Dock}', left=row.Start_Time,
width=row.Unload_Duration,
            color=row.Color, edgecolor='black')
    ax.text(row.Start_Time + row.Unload_Duration/2, f'Dock {row.Dock}',
            f'Ship {row.Ship} ({row.Priority})', va='center', ha='center',
fontsize=9)

ax.set_yticks([f'Dock {d}' for d in sorted(df_schedule['Dock'].unique())])
ax.set_yticklabels(dock_labels)  # ensure consistent order
ax.set_xlabel("Time")
ax.set_title("Docking Schedule with Priority Coloring (Sorted)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

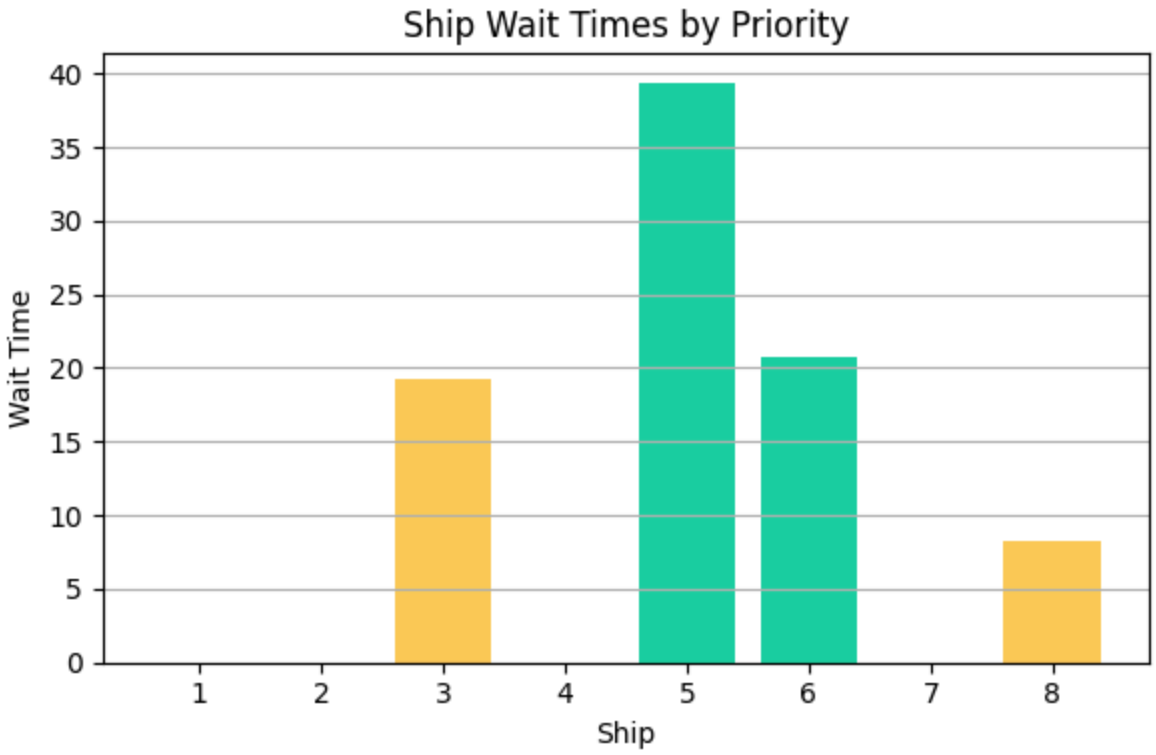
خروجی به شرح زیر خواهد بود:



همچنین براساس همان رنگ‌بندی می‌توان میزان معطلی هر کشتی را نیز به صورت نمودار میله‌ای رسم کرد، در پایین این کار را انجام می‌دهیم:

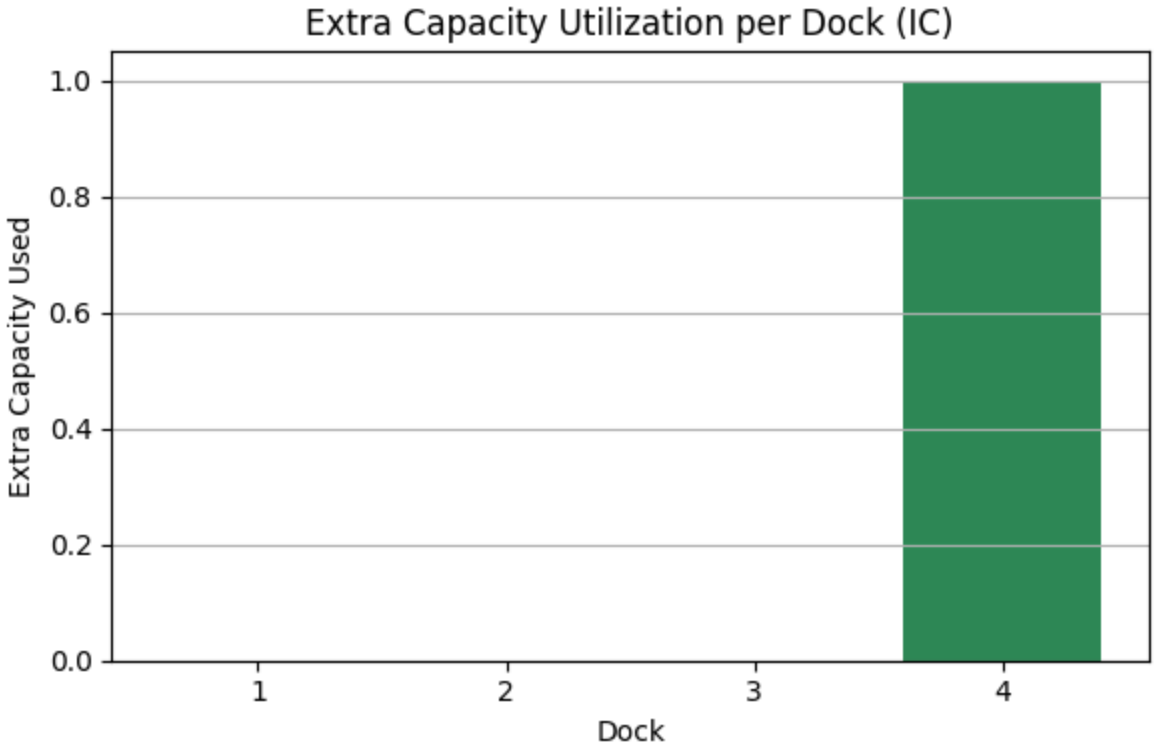
```
df_W = df_W.merge(df_priority, on="Ship")
```

```
fig, ax = plt.subplots(figsize=(6, 4))
colors = df_W['Priority'].map(priority_colors)
ax.bar(df_W['Ship'].astype(str), df_W['Wait_Time'], color=colors)
ax.set_xlabel("Ship")
ax.set_ylabel("Wait Time")
ax.set_title("Ship Wait Times by Priority")
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```



علاوه بر موارد فوق، با استفاده از قطعه کد و نمودار زیر می‌توانیم میزان افزایش ظرفیت هر اسکله را مشاهده کنیم:

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(df_IC['Dock'].astype(str), df_IC['Extra_Capacity_Used'],
color='seagreen')
ax.set_xlabel("Dock")
ax.set_ylabel("Extra Capacity Used")
ax.set_title("Extra Capacity Utilization per Dock (IC)")
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```



بخش چهارم) ریلکس کردن محدودیت اسکله سوم

تنها نکته متفاوت این بخش تغییر محدودیت مربوطه است، در اینجا محدودیت را تغییر می‌دهیم:

```
                                                    (model.del_component(model.c2
                                                :(def number_ships_to_pier(model, j
                                                                    :if j==3
                                                        return pyo.Constraint.Skip
                                                                                :else
                                    return pyo.quicksum(model.X[i,j] for i in model.i) <=2
                                (model.c2 = pyo.Constraint(model.j, rule= number_ships_to_pier
```

در ادامه روندی که در بخش‌های قبل طی کردیم را تکرار می‌کنیم، پاسخ مدل و نمودار گانت نیز در انتها آمده اند

```
solver= pyo.SolverFactory('glpk')
result = solver.solve(model, tee=True)
result.write()
new_obj = value(model.obj)
print(f"Original Objective: {original_obj}, New Objective: {new_obj}")
if result.solver.termination_condition != TerminationCondition.infeasible:
    for i, j in model.X.index_set():
        if model.X[i, j].value > 0:
            print(f"X[{i}, {j}] = {model.X[i,j].value}")
    for i in model.W.index_set():
        if model.W[i].value >= 0:
            print(f"W[{i}] = {model.W[i].value}")
    for i in model.s.index_set():
        if model.s[i].value >= 0:
            print(f"s[{i}] = {model.s[i].value}")

    for i,q, j in model.y.index_set():
        if model.y[i,q,j].value == 1:
            print(f"y[{i},{q}, {j}] = {model.y[i,q,j].value}")

    for i,j in model.u.index_set():
        if model.u[i,j].value > 0:
            print(f"u[{i},{j}] = {model.u[i,j].value}")


    obj_val = value(model.obj)
    print(f"Objective value: {obj_val}")
if result.solver.termination_condition != TerminationCondition.infeasible:
    # Export X[i, j]
    X_data = [(i, j, model.X[i, j].value) for (i, j) in model.X if model.X[i,
j].value is not None and model.X[i, j].value > 0]
    df_X = pd.DataFrame(X_data, columns=['Ship', 'Dock', 'Assigned'])

    # Export W[i]
    W_data = [(i, model.W[i].value) for i in model.W if model.W[i].value is not
None]
    df_W = pd.DataFrame(W_data, columns=['Ship', 'Wait_Time'])

    # Export s[i]
    s_data = [(i, model.s[i].value) for i in model.s if model.s[i].value is not
None]
    df_s = pd.DataFrame(s_data, columns=['Ship', 'Start_Time'])

    # Export y[i, q, j]
```

```
    y_data = [(i, q, j, model.y[i, q, j].value) for (i, q, j) in model.y if
model.y[i, q, j].value == 1]
    df_y = pd.DataFrame(y_data, columns=['Ship1', 'Ship2', 'Dock', 'y_value'])

    # Export u[i, k]
    u_data = [(i, k, model.u[i, k].value) for (i, k) in model.u if model.u[i,
k].value is not None and model.u[i, k].value > 0]
    df_u = pd.DataFrame(u_data, columns=['Ship', 'Objective_K', 'u_value'])

    # Export IC[j]
    IC_data = [(j, model.IC[j].value) for j in model.IC ]
    df_IC = pd.DataFrame(IC_data, columns=['Dock', 'Extra_Capacity_Used'])

    #Exort Un[i,j]

    # Objective value
    obj_val = value(model.obj)

    # Save all to Excel
    with pd.ExcelWriter("pyomo_results_dock3_loosened.xlsx") as writer:
        df_X.to_excel(writer, sheet_name="X", index=False)
        df_W.to_excel(writer, sheet_name="W", index=False)
        df_s.to_excel(writer, sheet_name="s", index=False)
        df_y.to_excel(writer, sheet_name="y", index=False)
        df_u.to_excel(writer, sheet_name="u", index=False)
        df_IC.to_excel(writer, sheet_name="IC", index=False)

    print("Objective Value:", obj_val)
    print("Results saved to pyomo_results_dock3_loosened.xlsx")
df_Un = pd.DataFrame({'Ship': df_s['Ship'], 'Unload_Duration': 10})  # Replace
10 with actual Un[i,j]

priority_data = [(i, float(value(model.P[i]))) for i in model.P]
df_priority = pd.DataFrame(priority_data, columns=['Ship', 'Priority_Value'])

df_priority['Priority'] =
df_priority['Priority_Value'].apply(classify_priority)
# Merge everything
df_schedule = df_X.merge(df_s, on="Ship").merge(df_Un,
on="Ship").merge(df_priority, on="Ship")
df_schedule['End_Time'] = df_schedule['Start_Time'] +
df_schedule['Unload_Duration']
```

```
df_schedule['Color'] = df_schedule['Priority'].map(priority_colors)
# Make sure dock is numeric if it isn't already
df_schedule['Dock'] = df_schedule['Dock'].astype(int)

# Sort by Dock (numerically)
df_schedule = df_schedule.sort_values(by='Dock', ascending=False)

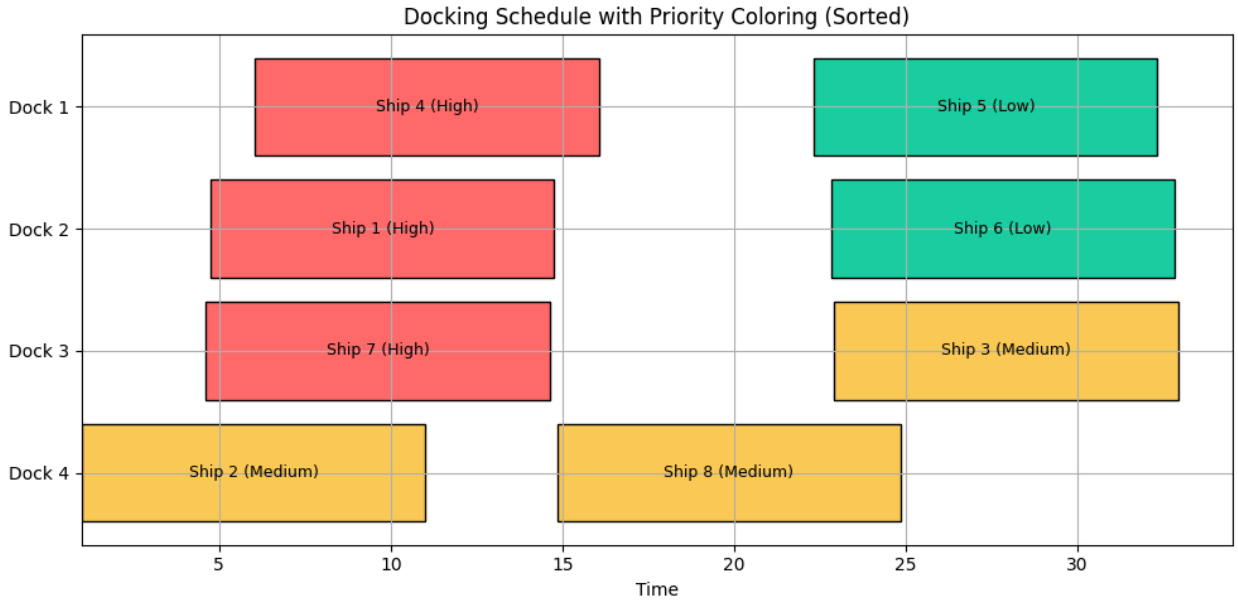
# Create an ordered list of dock labels
dock_labels = sorted(df_schedule['Dock'].unique())
dock_labels = [f'Dock {d}' for d in dock_labels]

# === Gantt Chart with sorted docks ===
fig, ax = plt.subplots(figsize=(10, 5))
for _, row in df_schedule.iterrows():
    ax.barh(y=f'Dock {row.Dock}', left=row.Start_Time,
width=row.Unload_Duration,
            color=row.Color, edgecolor='black')
    ax.text(row.Start_Time + row.Unload_Duration/2, f'Dock {row.Dock}',
            f'Ship {row.Ship} ({row.Priority})', va='center', ha='center',
fontsize=9)

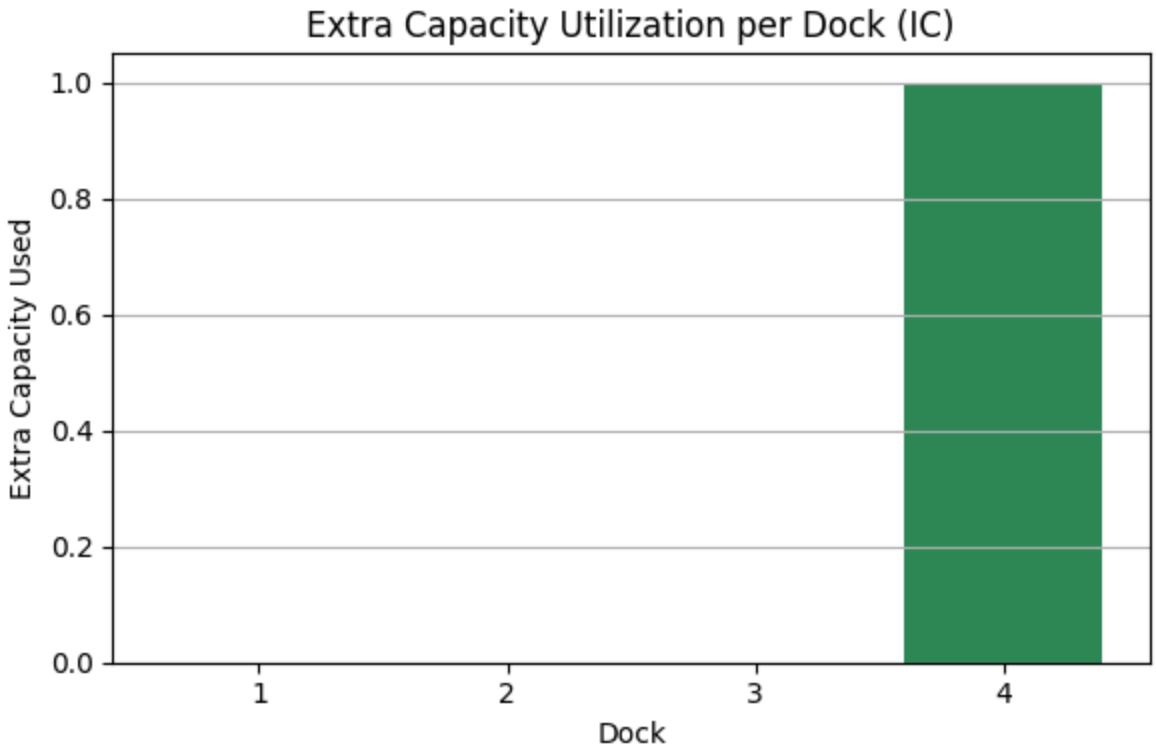
ax.set_yticks([f'Dock {d}' for d in sorted(df_schedule['Dock'].unique())])
ax.set_yticklabels(dock_labels)  # ensure consistent order
ax.set_xlabel("Time")
ax.set_title("Docking Schedule with Priority Coloring (Sorted)")
plt.grid(True)
plt.tight_layout()
plt.show()
df_W = df_W.merge(df_priority, on="Ship")
fig, ax = plt.subplots(figsize=(6, 4))
colors = df_W['Priority'].map(priority_colors)
ax.bar(df_W['Ship'].astype(str), df_W['Wait_Time'], color=colors)
ax.set_xlabel("Ship")
ax.set_ylabel("Wait Time")
ax.set_title("Ship Wait Times by Priority")
plt.tight_layout()
plt.grid(axis='y')
plt.show()
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(df_IC['Dock'].astype(str), df_IC['Extra_Capacity_Used'],
color='seagreen')
ax.set_xlabel("Dock")
ax.set_ylabel("Extra Capacity Used")
```

```
ax.set_title("Extra Capacity Utilization per Dock (IC)")
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```

سه نموداری که پیشتر شرح دادیم به شکل زیر خواهند بود:




Ship Wait Times by Priority
Wait Time
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
1
2
3
4
5
6
7
8
Ship


Extra Capacity Utilization per Dock (IC)
Extra Capacity Used
0.0
0.2
0.4
0.6
0.8
1.0
1
2
3
4
Dock

همچنین نتایج مدل برای بررسی دقیق‌تر به شرح زیر خواهند بود:

```
X[1, 2] = 1.0
X[2, 4] = 1.0
X[3, 3] = 1.0
X[4, 1] = 1.0
X[5, 1] = 1.0
X[6, 2] = 1.0
X[7, 3] = 1.0
X[8, 4] = 1.0
W[1] = 0.0
W[2] = 0.0
W[3] = 19.2670572909898
W[4] = 0.0
W[5] = 20.457012660854
W[6] = 20.6958212309185
W[7] = 0.0
W[8] = 8.24083515240811
s[1] = 4.76552698703798
s[2] = 1.00459250301696
s[3] = 22.9200559414305
s[4] = 6.05417812493725
s[5] = 22.3262021407276
s[6] = 22.813811450173
s[7] = 4.62295182452406
s[8] = 14.8420135358433
y[1,6, 2] = 1.0
y[2,8, 4] = 1.0
y[4,5, 1] = 1.0
y[7,3, 3] = 1.0
u[3,1] = 10.0
u[3,2] = 9.26705729098979
u[5,1] = 10.0
```

```
u[5,2] = 10.457012660854
u[6,1] = 10.0
u[6,2] = 10.6958212309185
u[8,1] = 8.24083515240811
Objective value: 1605.5574635658484
```

بخش ششم) تحلیل حساسیت

در ابتدا مقادیر اصلی پارامترها را ذخیره می‌کنیم:

```
original_o2_value = pyo.value(model.O[2])
original_Un_7_3_value = pyo.value(model.Un[7, 3])
original_C_2_value = pyo.value(model.C[2])
original_C_3_value = pyo.value(model.C[3])
original_a_6_value = pyo.value(model.a[6])
```

زمان بین تخلیه دو کشتی در اسکله دو)

با استفاده از یک حلقه مقدار پارامتر اول یعنی زمان بین دو تخلیه در اسکله 2 را تغییر می‌دهیم و مقادیر متناظر پارامترها را در یک دیتافریم ذخیره می‌کنیم تا در مراحل بعدی مصورسازی کنیم:

```
import pandas as pd
import matplotlib.pyplot as plt
from pyomo.environ import value

  # Store the original value of O[2]
# Step range: 5 to 10 with 0.5 increments
o2_values = [round(v * 0.5, 2) for v in range(10, 21)]  # Gives values from 5.0
to 10.0 in 0.5 steps
results = []
o2_values.insert(0, original_o2_value)  # Append the original value to the list
# Loop through each value for O[2]
for val in o2_values :
    model.O[2] = val  # Update the parameter

    # Solve the model
    solver = pyo.SolverFactory('glpk')
    result = solver.solve(model)

    # Record objective
    obj_val = value(model.obj)
```

```
    # Record decision variable x[i,j]
    x_vals = {(i, j): value(model.X[i, j]) for i in model.i for j in model.j if
model.X[i, j].value >0}
    x_vals = {(f'x[{i},{j}]'): value(model.X[i, j]) for i in model.i for j in
model.j if model.X[i,j].value>0}
    W_vals = {(f'W[{i}]'): value(model.W[i]) for i in model.i}
    IC_vals = {(f'IC[{j}]'): value(model.IC[j]) for j in model.j}
    s_vals = {(f's[{i}]'): value(model.s[i]) for i in model.i}
    Un_vals = {(f'Un[{i},{j}]'): value(model.Un[i,j]) for i in model.i for j in
model.j}

    # Flatten results into one row per (o2_val)
    row = {
        'o2_value': val,
        'objective': obj_val,
        'x_values': x_vals,
        'W_vals': W_vals,
        'IC_vals': IC_vals,
        's_vals': s_vals,
        'Un_vals': Un_vals
    }
    results.append(row)

# Convert to DataFrame
df_results = pd.DataFrame(results)

# Expand x_values into columns
df_x = df_results['x_values'].apply(pd.Series)
df_W = df_results['W_vals'].apply(pd.Series)
df_IC = df_results['IC_vals'].apply(pd.Series)
df_s = df_results['s_vals'].apply(pd.Series)
df_Un = df_results['Un_vals'].apply(pd.Series)
df_final = pd.concat([df_results[['o2_value', 'objective']], df_x,df_W,df_IC],
axis=1)
df_final
```

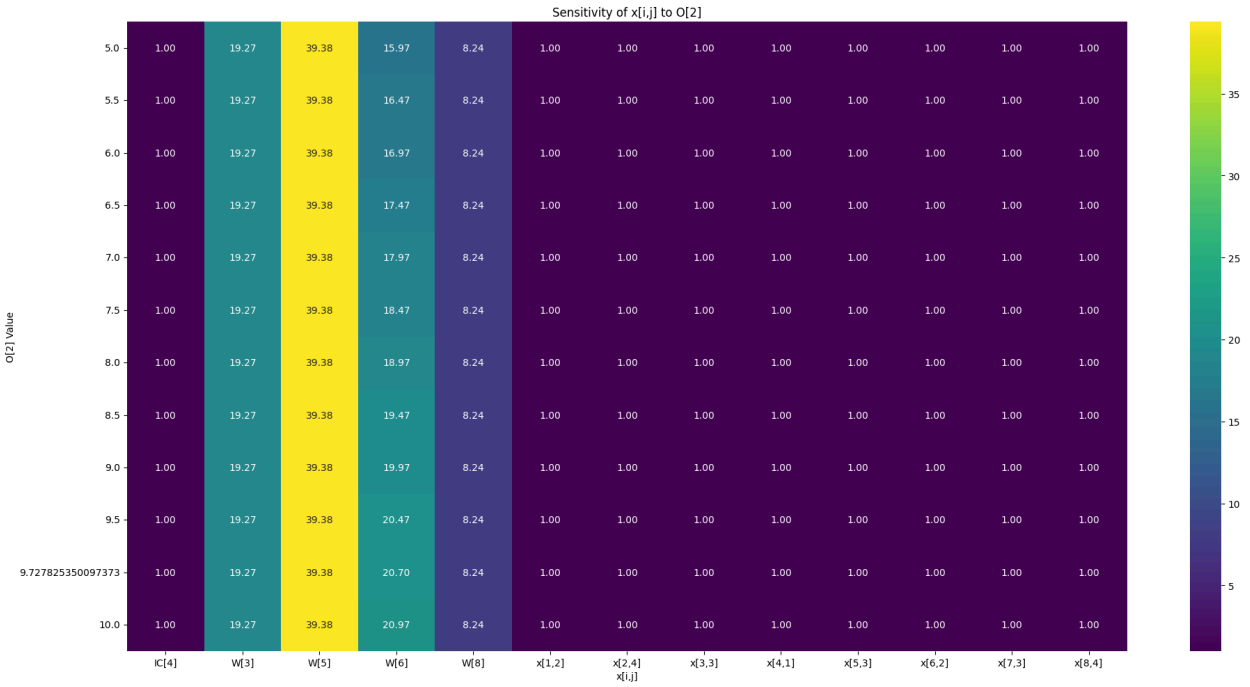
در کد اصلی نمودارهای مختلفی رسم شده‌اند ولی دو نمودار که بیشترین انتقال داده را دارند را در اینجا آورده‌ایم:

```
# Convert wide-form df to long-form
df_long = df_final.melt(id_vars=['o2_value', 'objective'], var_name='(i,j)',
value_name='x_val')

# Filter only x variables that changed meaningfully (optional)
df_long = df_long[df_long['x_val'].abs() > 1e-3]

# Heatmap of decision variable changes
pivot = df_long.pivot(index='o2_value', columns='(i,j)', values='x_val')
plt.figure(figsize=(20, 10))
sns.heatmap(pivot, cmap="viridis", annot=True, fmt=".2f")
plt.title("Sensitivity of x[i,j] to O[2]")
plt.xlabel("x[i,j]")
plt.ylabel("O[2] Value")
plt.tight_layout()
plt.show()
```

نمودار حاصل از قطعه کد بالا به شرح زیر خواهد بود:



که انتظار داشتیم تخصیص‌ها وابسته به این پارامتر نباشند و صرفا زمان‌های انتظار تغییر کنند.
برای رسم نمودار دوم داریم:

```
# Set Seaborn style
sns.set_style("whitegrid")
plt.rcParams.update({'font.size': 11, 'axes.labelweight': 'bold',
```

```
'axes.titlesize': 14})

# Define colors and marker styles
main_color = 'steelblue'
highlight_color = 'crimson'
marker_style = 'o'
marker_size = 8
highlight_size = 80

# Start plot
plt.figure(figsize=(14, 10))

# Original values
orig_idx = 0
orig_x = df_final['o2_value'].iloc[orig_idx]
orig_obj = df_final['objective'].iloc[orig_idx]

# 1. Objective vs o2_value
plt.subplot(2, 2, 1)
sns.lineplot(data=df_final, x='o2_value', y='objective', marker=marker_style,
             color=main_color, markersize=marker_size, linewidth=2)
plt.scatter(orig_x, orig_obj, color=highlight_color, s=highlight_size,
label='Original Point', zorder=5)
plt.title('Objective Function vs o2_value')
plt.xlabel('o2_value')
plt.ylabel('Objective Value')
plt.legend()

# 2. Percentage change from original
df_final['pct_change'] = ((df_final['objective'] - orig_obj) / orig_obj) * 100

plt.subplot(2, 2, 2)
sns.lineplot(data=df_final, x='o2_value', y='pct_change', marker=marker_style,
             color='darkorange', markersize=marker_size, linewidth=2)
plt.scatter(orig_x, df_final['pct_change'].iloc[orig_idx],
color=highlight_color, s=highlight_size, zorder=5)
plt.axhline(0, color='gray', linestyle='--', alpha=0.5)
plt.title('Percentage Change from Original')
plt.xlabel('o2_value')
plt.ylabel('Change (%)')

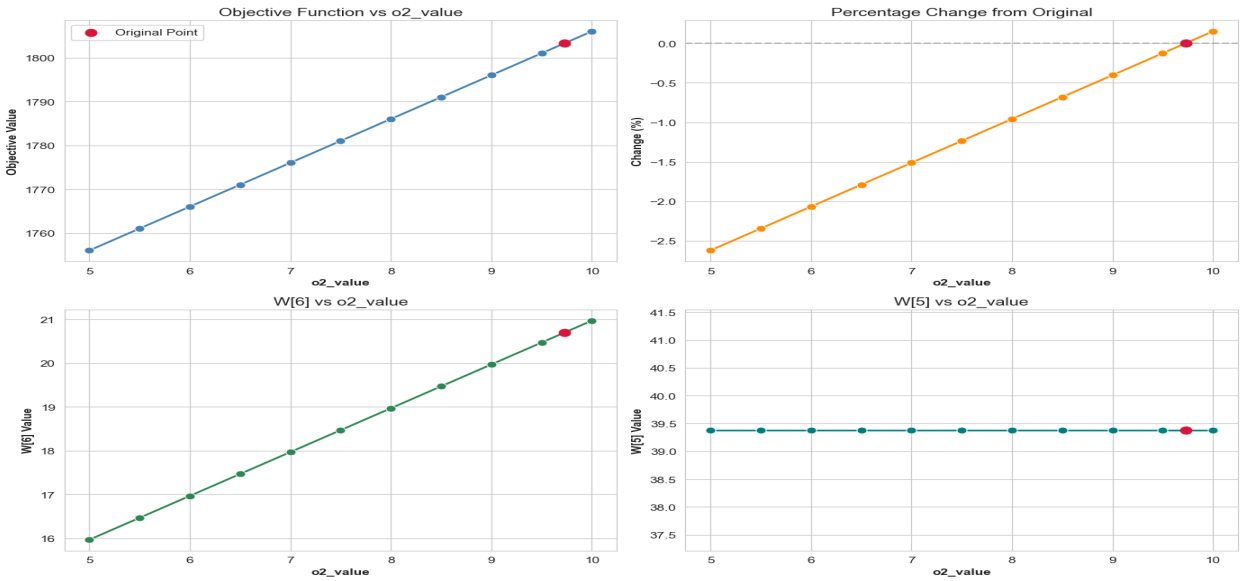
# 3. W[6] vs o2_value (if exists)
if 'W[6]' in df_final.columns:
    plt.subplot(2, 2, 3)
```

```
    sns.lineplot(data=df_final, x='o2_value', y='W[6]', marker=marker_style,
                 color='seagreen', markersize=marker_size, linewidth=2)
    plt.scatter(orig_x, df_final['W[6]'].iloc[orig_idx], color=highlight_color,
s=highlight_size, zorder=5)
    plt.title('W[6] vs o2_value')
    plt.xlabel('o2_value')
    plt.ylabel('W[6] Value')

# 4. W[5] vs o2_value (if exists)
if 'W[5]' in df_final.columns:
    plt.subplot(2, 2, 4)
    sns.lineplot(data=df_final, x='o2_value', y='W[5]', marker=marker_style,
                 color='teal', markersize=marker_size, linewidth=2)
    plt.scatter(orig_x, df_final['W[5]'].iloc[orig_idx], color=highlight_color,
s=highlight_size, zorder=5)
    plt.title('W[5] vs o2_value')
    plt.xlabel('o2_value')
    plt.ylabel('W[5] Value')

plt.tight_layout()
plt.show()
```

خروجی به قرار زیر است:



این نمودار هم نشان می‌دهد که زمانهای انتظار و تابع هدف با پارامتر مذکور رابطه مستقیم دارند.

همچنین می‌توان یک دیتا فریم خلاصه از میزان حساسیت متغیرها تهیه کرد:

```
if len(df_final) > 1:
    comparison_df = pd.DataFrame({
        'Variable': ['objective', 'W[6]', 'W[5]'],
        'Original Value': [df_final['objective'].iloc[0],
df_final['W[6]'].iloc[0], df_final['W[5]'].iloc[0]],
        'Min Value': [df_final['objective'].min(), df_final['W[6]'].min(),
df_final['W[5]'].min()],
        'Max Value': [df_final['objective'].max(), df_final['W[6]'].max(),
df_final['W[5]'].max()],
        'Range': [df_final['objective'].max() - df_final['objective'].min(),
                  df_final['W[6]'].max() - df_final['W[6]'].min(),
                  df_final['W[5]'].max() - df_final['W[5]'].min()],
        'Sensitivity (%)': [
            abs((df_final['objective'].max() - df_final['objective'].min()) /
df_final['objective'].iloc[0]) * 100,
            abs((df_final['W[6]'].max() - df_final['W[6]'].min()) /
df_final['W[6]'].iloc[0]) * 100,
            abs((df_final['W[5]'].max() - df_final['W[5]'].min()) /
df_final['W[5]'].iloc[0]) * 100
        ]
    })

    styled_df = comparison_df.style \
        .set_properties(**{'font-weight': 'bold'}, subset=['Variable']) \
        .background_gradient(cmap='YlGnBu', subset=['Sensitivity (%)']) \
        .format({
            'Original Value': '{:.4f}',
            'Min Value': '{:.4f}',
            'Max Value': '{:.4f}',
            'Range': '{:.4f}',
            'Sensitivity (%)': '{:.2f}%'
        })

    print("\nSensitivity Analysis Summary:")
    display(styled_df)
```

خروجی به شرح زیر است:(میزان معطلی کشتی شش بیشترین حساسیت را به این پارامتر دارد)

	Variable	Original Value	Min Value	Max Value	Range	Sensitivity (%)
0	objective	1803.2706	1755.9923	1805.9923	50.0000	2.77%
1	W[6]	20.6958	15.9680	20.9680	5.0000	24.16%
2	W[5]	39.3789	39.3789	39.3789	0.0000	0.00%

زمان تخلیه کشتی شماره هفت روی اسکله شماره سه)

پس از تنظیم مقادیر پارامترها به مقادیر اولیه، مانند بخش قبل ابتدا با استفاده از یک حلقه مقدار پارامتر مربوطه را تغییر می‌دهیم و نتایج مدل را در یک دیتافریم ذخیره می‌کنیم:

```
Un_7_3_values = [round(v * 0.5, 2) for v in range(10, 31)]  # Gives values from
5.0 to 10.0 in 0.5 steps
results = []
Un_7_3_values.insert(0, original_Un_7_3_value)  # Append the original value to
the list
# Loop through each value for o[3]
for val in Un_7_3_values :
    model.Un[7,3] = val  # Update the parameter

    # Solve the model
    solver = pyo.SolverFactory('glpk')
    result = solver.solve(model)

    # Record objective
    obj_val = value(model.obj)

    # Record decision variable x[i,j]
    x_vals = {(i, j): value(model.X[i, j]) for i in model.i for j in model.j if
model.X[i, j].value >0}
    x_vals = {(f'x[{i},{j}]'): value(model.X[i, j]) for i in model.i for j in
model.j if model.X[i,j].value>0}
    W_vals = {(f'W[{i}]'): value(model.W[i]) for i in model.i}
    IC_vals = {(f'IC[{j}]'): value(model.IC[j]) for j in model.j}
    s_vals = {(f's[{i}]'): value(model.s[i]) for i in model.i}
    Un_vals = {(f'Un[{i},{j}]'): value(model.Un[i,j]) for i in model.i for j in
model.j}

    # Flatten results into one row per (o3_val)
    row = {
        'Un_7_3_value': val,
        'objective': obj_val,
        'x_values': x_vals,
        'W_vals': W_vals,
        'IC_vals': IC_vals,
        's_vals': s_vals,
        'Un_vals': Un_vals
    }
    results.append(row)
```

```
# Convert to DataFrame
df_results = pd.DataFrame(results)

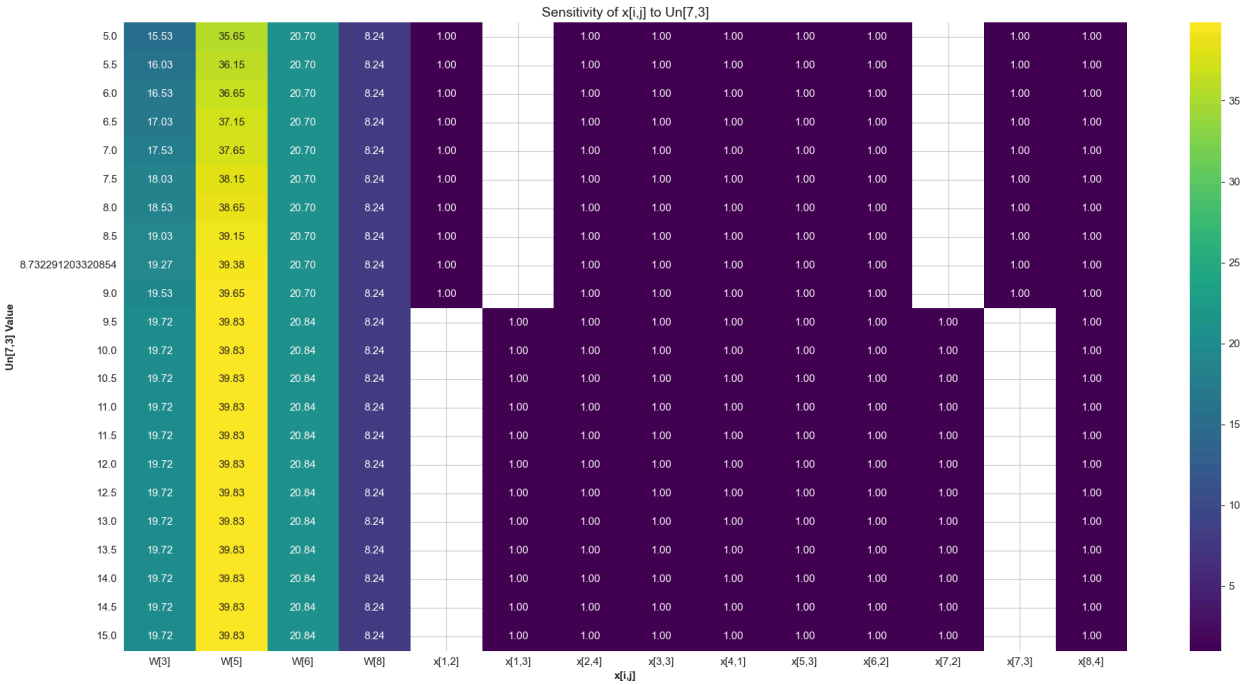
# Expand x_values into columns
df_x = df_results['x_values'].apply(pd.Series)
df_W = df_results['W_vals'].apply(pd.Series)
df_IC = df_results['IC_vals'].apply(pd.Series)
df_s = df_results['s_vals'].apply(pd.Series)
df_Un = df_results['Un_vals'].apply(pd.Series)
df_final = pd.concat([df_results[['Un_7_3_value', 'objective']], df_x,df_W],
axis=1)
df_final
```

مانند بخش قبل دو نمودار رسم می‌کنیم، برای رسم هیت‌مپ داریم:

```
# Convert wide-form df to long-form
df_long = df_final.melt(id_vars=['Un_7_3_value',
'objective'], var_name='(i,j)', value_name='x_val')

# Filter only x variables that changed meaningfully
(optional)
df_long = df_long[df_long['x_val'].abs() > 1e-3]

# Heatmap of decision variable changes
pivot = df_long.pivot(index='Un_7_3_value',
columns='(i,j)', values='x_val')
plt.figure(figsize=(20, 10))
sns.heatmap(pivot, cmap="viridis", annot=True, fmt=".2f")
plt.title("Sensitivity of x[i,j] to Un[7,3]")
plt.xlabel("x[i,j]")
plt.ylabel("Un[7,3] Value")
plt.tight_layout()
plt.show()
```



مانند بخش قبل مشاهده می‌شود که تخصیص‌ها (تا مقدار مشخصی از تغییر) ارتباطی با این پارامتر ندارند و صرفا زمان‌های انتظار تغییر می‌کنند، برای بررسی بیشتر داریم:

```
# Style settings
sns.set_style("whitegrid")
plt.rcParams.update({
    'font.size': 11,
    'axes.labelweight': 'bold',
    'axes.titlesize': 14
})

# Colors & markers
main_color = 'steelblue'
highlight_color = 'crimson'
marker_style = 'o'
marker_size = 6
highlight_size = 100

# Original point index
orig_idx = 0
orig_x = df_final['Un_7_3_value'].iloc[orig_idx]
orig_obj = df_final['objective'].iloc[orig_idx]

# Calculate percentage change
df_final['pct_change'] = ((df_final['objective'] - orig_obj) / orig_obj) * 100
```

```
# List of variable plots (name, label)
variables = [
    ('objective', 'Objective Value'),
    ('pct_change', 'Change (%)'),
]

# Add W[i] columns if present
for w_col in ['W[3]', 'W[5]', 'W[6]']:
    if w_col in df_final.columns:
        variables.append((w_col, f'{w_col} Value'))

# Decision switches: add flags and labels
decision_plots = []

if {'x[1,2]', 'x[1,3]'}.issubset(df_final.columns):
    decision_plots.append(('x[1,2]', 'x[1,3]', 'Decision x[1,j]'))

if {'x[7,2]', 'x[7,3]'}.issubset(df_final.columns):
    decision_plots.append(('x[7,2]', 'x[7,3]', 'Decision x[7,j]'))

# Add decision switch placeholders to variable list
for _, _, label in decision_plots:
    variables.append((label, 'Chosen Decision'))

# Layout
num_plots = len(variables)
cols = 2
rows = math.ceil(num_plots / cols)

# Start plotting
plt.figure(figsize=(7 * cols, 4.5 * rows))

for i, (var, ylabel) in enumerate(variables, 1):
    plt.subplot(rows, cols, i)

    if var.startswith("Decision"):
        # Find correct pair of X variables
        for x1, x2, label in decision_plots:
            if label == var:
                break

        # Create long-form DataFrame
        df_x = df_final[['Un_7_3_value', x1, x2]].copy()
```

```
        df_x['Index'] = df_x.index
        df_long = df_x.melt(id_vars=['Un_7_3_value', 'Index'],
                            value_vars=[x1, x2],
                            var_name='Decision', value_name='Chosen')
        df_long = df_long[df_long['Chosen'] == 1]

        # Plot decision switch
        sns.scatterplot(data=df_long, x='Un_7_3_value', y='Decision',
hue='Decision',
                        style='Decision', s=80,
                        palette={x1: 'royalblue', x2: 'darkorange'})

        # Highlight original point
        orig_dec = df_long[df_long['Index'] == orig_idx]
        plt.scatter(orig_dec['Un_7_3_value'], orig_dec['Decision'],
                    color=highlight_color, s=highlight_size,
                    edgecolor='black', label='Original Point', zorder=5)

        plt.title(f'{label} vs Un_7_3_value')
        plt.ylabel('Chosen Decision Variable')
        plt.xlabel('Un_7_3_value')
        plt.legend()

    else:
        # Standard continuous plot
        color = 'darkorange' if var == 'pct_change' else main_color
        if 'W[' in var:
            color = 'teal'

        sns.lineplot(data=df_final, x='Un_7_3_value', y=var,
                     marker=marker_style, color=color,
                     markersize=marker_size, linewidth=2)

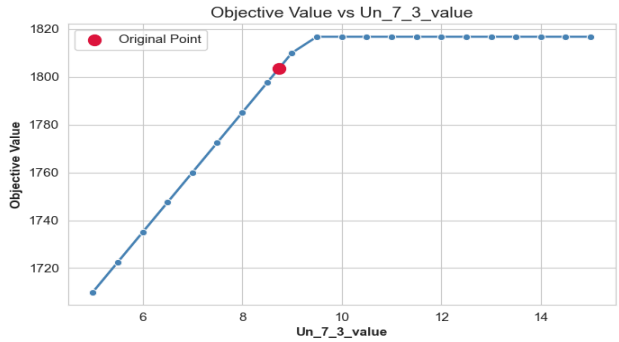
        # Highlight original
        plt.scatter(orig_x, df_final[var].iloc[orig_idx],
                    color=highlight_color, s=highlight_size,
                    label='Original Point', zorder=5)

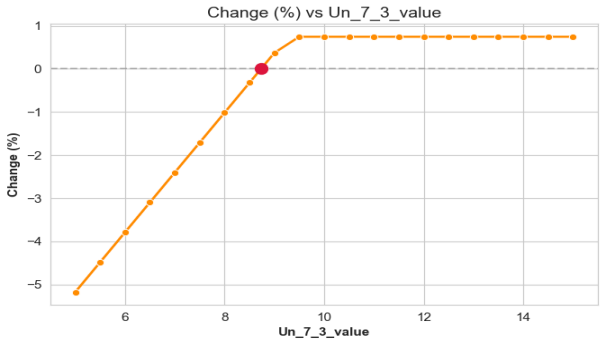
        if var == 'pct_change':
            plt.axhline(0, color='gray', linestyle='--', alpha=0.5)

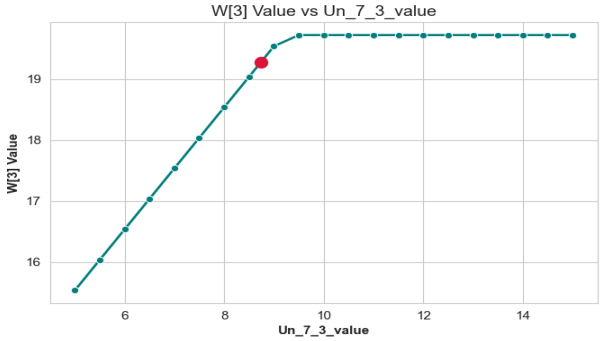
        plt.title(f'{ylabel} vs Un_7_3_value')
        plt.xlabel('Un_7_3_value')
        plt.ylabel(ylabel)
```

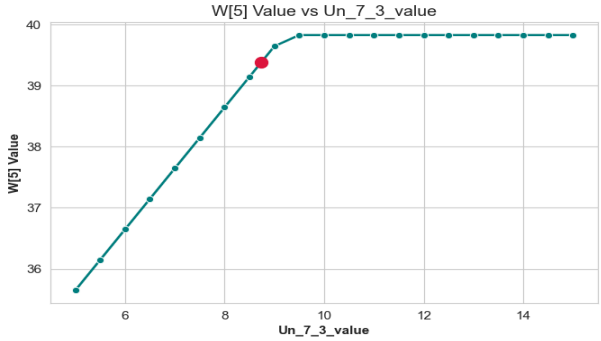
```
        if i == 1:
            plt.legend()

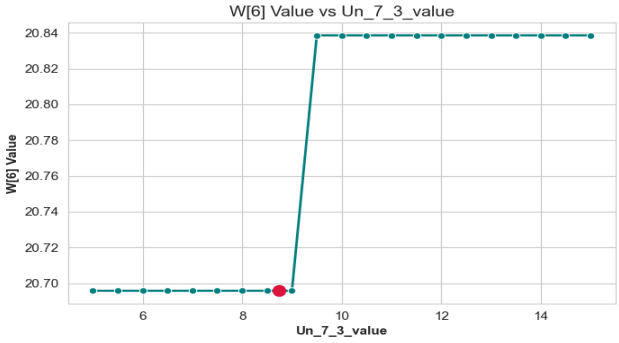
plt.tight_layout()
plt.show()
```

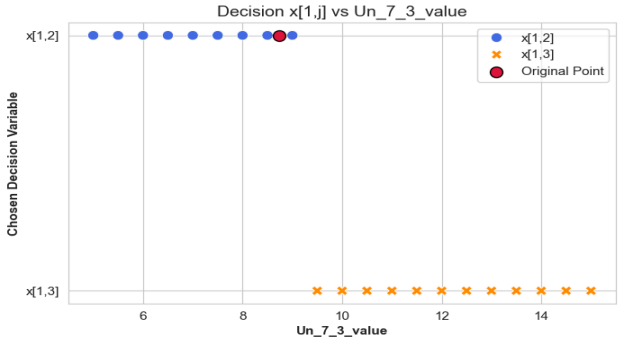


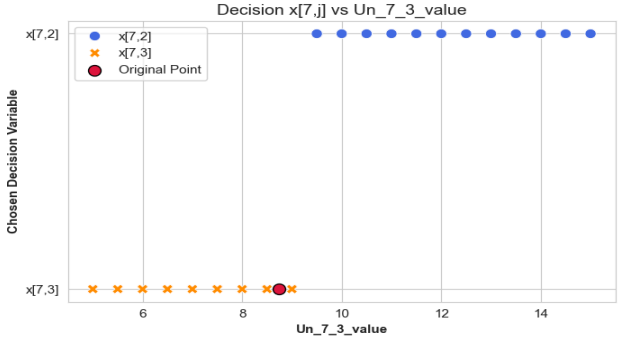












مشاهده می‌شود که با افزایش بیش از یک حد مشخص این پارامتر، تخصیص‌ها تغییر می‌کنند و در سایر مقادیر تا همان حد مشخص تخصیص‌ها بلاتغییر و مقادیر انتظار و تابع هدف با پارامتر مربوطه رابطه خطی دارند.

ظرفیت اسکله شماره دو)

در این بخش همانند بخش قبل مقدار پارامتر را به ترتیب تغییر داده و در دیتافریم ذخیره می‌کنیم:

```
C_2_values = [round(v * 1, 2) for v in range(5, 40)]  # Gives values from
5.0 to 80.0 in 0.5 steps
results = []
C_2_values.insert(0, original_C_2_value)  # Append the original value to
the list
# Loop through each value for C[2]
for val in C_2_values :
    model.C[2] = val  # Update the parameter

    # Solve the model
    solver = pyo.SolverFactory('glpk')
    result = solver.solve(model)

    # Record objective
    obj_val = value(model.obj)

    # Record decision variable x[i,j]
      x_vals = {(i, j): value(model.X[i, j]) for i in model.i for j in
model.j if model.X[i, j].value >0}
    x_vals = {(f'x[{i},{j}]'): value(model.X[i, j]) for i in model.i for j
in model.j if model.X[i,j].value>0}
    W_vals = {(f'W[{i}]'): value(model.W[i]) for i in model.i}
    IC_vals = {(f'IC[{j}]'): value(model.IC[j]) for j in model.j}
    s_vals = {(f's[{i}]'): value(model.s[i]) for i in model.i}
    C_vals = {(f'C[{j}]'): value(model.C[j]) for j in model.j}

    # Flatten results into one row per (o3_val)
```

```
    row = {
        'C_2_value': val,
        'objective': obj_val,
        'x_values': x_vals,
        'W_vals': W_vals,
        'IC_vals': IC_vals,
        's_vals': s_vals,
        'C_vals': C_vals
    }
    results.append(row)

# Convert to DataFrame
df_results = pd.DataFrame(results)

# Expand x_values into columns
df_x = df_results['x_values'].apply(pd.Series)
df_W = df_results['W_vals'].apply(pd.Series)
df_IC = df_results['IC_vals'].apply(pd.Series)
df_s = df_results['s_vals'].apply(pd.Series)
df_C = df_results['C_vals'].apply(pd.Series)
df_final      =      pd.concat([df_results[['C_2_value',      'objective']],
df_x,df_W,df_IC], axis=1)
df_final
```

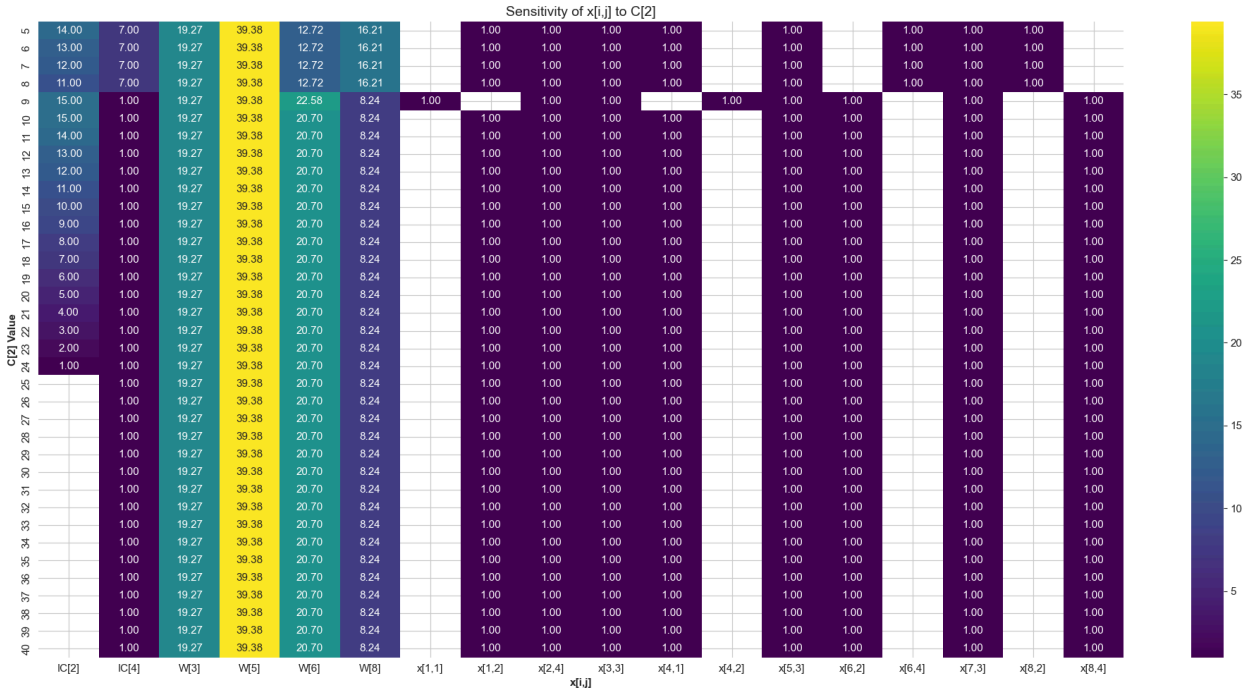
مانند بخش قبل دو نمودار رسم می‌کنیم، برای رسم هیت‌مپ داریم:

```
# Convert wide-form df to long-form
df_long      =      df_final.melt(id_vars=['C_2_value',      'objective'],
var_name='(i,j)', value_name='x_val')

# Filter only x variables that changed meaningfully (optional)
df_long = df_long[df_long['x_val'].abs() > 1e-3]

# Heatmap of decision variable changes
pivot = df_long.pivot(index='C_2_value', columns='(i,j)', values='x_val')
plt.figure(figsize=(20, 10))
sns.heatmap(pivot, cmap="viridis", annot=True, fmt=".2f")
```

```
plt.title("Sensitivity of x[i,j] to C[2]")
plt.xlabel("x[i,j]")
plt.ylabel("C[2] Value")
plt.tight_layout()
plt.show()
```



در این هیت مپ پس از مقدار 9 تخصیص کشتی‌های 1و 4 برای یک حالت تغییر می‌کند و برای کشتی 6 و 8 پس از مقدار 9 هر دو از اسکله 2 به اسکله 4 تخصیص می‌یابند.

حال برای دیدن تاثیرات این تغییر در تابع هدف داریم:

```
# Style settings
sns.set_style("whitegrid")
plt.rcParams.update({
    'font.size': 11,
    'axes.labelweight': 'bold',
    'axes.titlesize': 14
})

# Colors & markers
main_color = 'steelblue'
highlight_color = 'crimson'
marker_style = 'o'
```

```
marker_size = 6
highlight_size = 100

# Original point index
orig_idx = 0
orig_x = df_final['C_2_value'].iloc[orig_idx]
orig_obj = df_final['objective'].iloc[orig_idx]

# Calculate percentage change
df_final['pct_change'] = ((df_final['objective'] - orig_obj) / orig_obj) *
100

# Base continuous plots
variables = [
    ('objective', 'Objective Value'),
    ('pct_change', 'Change (%)'),
]

# Add W[i] and IC[i] variables if present
for col in ['W[3]', 'W[5]', 'W[6]', 'W[8]', 'IC[2]', 'IC[4]']:
    if col in df_final.columns:
        variables.append((col, f'{col} Value'))

# Decision groups to check
decision_groups = {
    'Decision x[1,j]': ['x[1,1]', 'x[1,2]'],
    'Decision x[4,j]': ['x[4,1]', 'x[4,2]'],
    'Decision x[6,j]': ['x[6,2]', 'x[6,4]'],
    'Decision x[7,j]': ['x[7,2]', 'x[7,3]'],
    'Decision x[8,j]': ['x[8,2]', 'x[8,4]'],
}

# Filter valid decision groups
decision_plots = []
for label, cols in decision_groups.items():
    if all(col in df_final.columns for col in cols):
```

```
        decision_plots.append((cols, label))
        variables.append((label, 'Chosen Decision'))

# Layout
num_plots = len(variables)
cols = 2
rows = math.ceil(num_plots / cols)

# Plotting
plt.figure(figsize=(7 * cols, 4.5 * rows))

for i, (var, ylabel) in enumerate(variables, 1):
    plt.subplot(rows, cols, i)

    if var.startswith("Decision"):
        # Get correct group of columns
        for group_cols, label in decision_plots:
            if label == var:
                break

        # Prepare long-form DataFrame
        df_x = df_final[['C_2_value'] + group_cols].copy()
        df_x['Index'] = df_x.index
        df_long = df_x.melt(id_vars=['C_2_value', 'Index'],
                            value_vars=group_cols,
                            var_name='Decision', value_name='Chosen')
        df_long = df_long[df_long['Chosen'] == 1]

        # Plot switches
        sns.scatterplot(data=df_long, x='C_2_value', y='Decision',
                        hue='Decision', style='Decision', s=80)

        # Highlight original
        orig_dec = df_long[df_long['Index'] == orig_idx]
        plt.scatter(orig_dec['C_2_value'], orig_dec['Decision'],
                    color=highlight_color, s=highlight_size,
```

```
                    edgecolor='black', label='Original Point', zorder=5)

        plt.title(f'{label} vs C_2_value')
        plt.ylabel('Chosen Decision Variable')
        plt.xlabel('C_2_value')
        plt.legend()

    else:
        # Continuous line plots
        color = 'darkorange' if var == 'pct_change' else main_color
        if 'W[' in var or 'IC[' in var:
            color = 'teal'

        sns.lineplot(data=df_final, x='C_2_value', y=var,
                     marker=marker_style, color=color,
                     markersize=marker_size, linewidth=2)

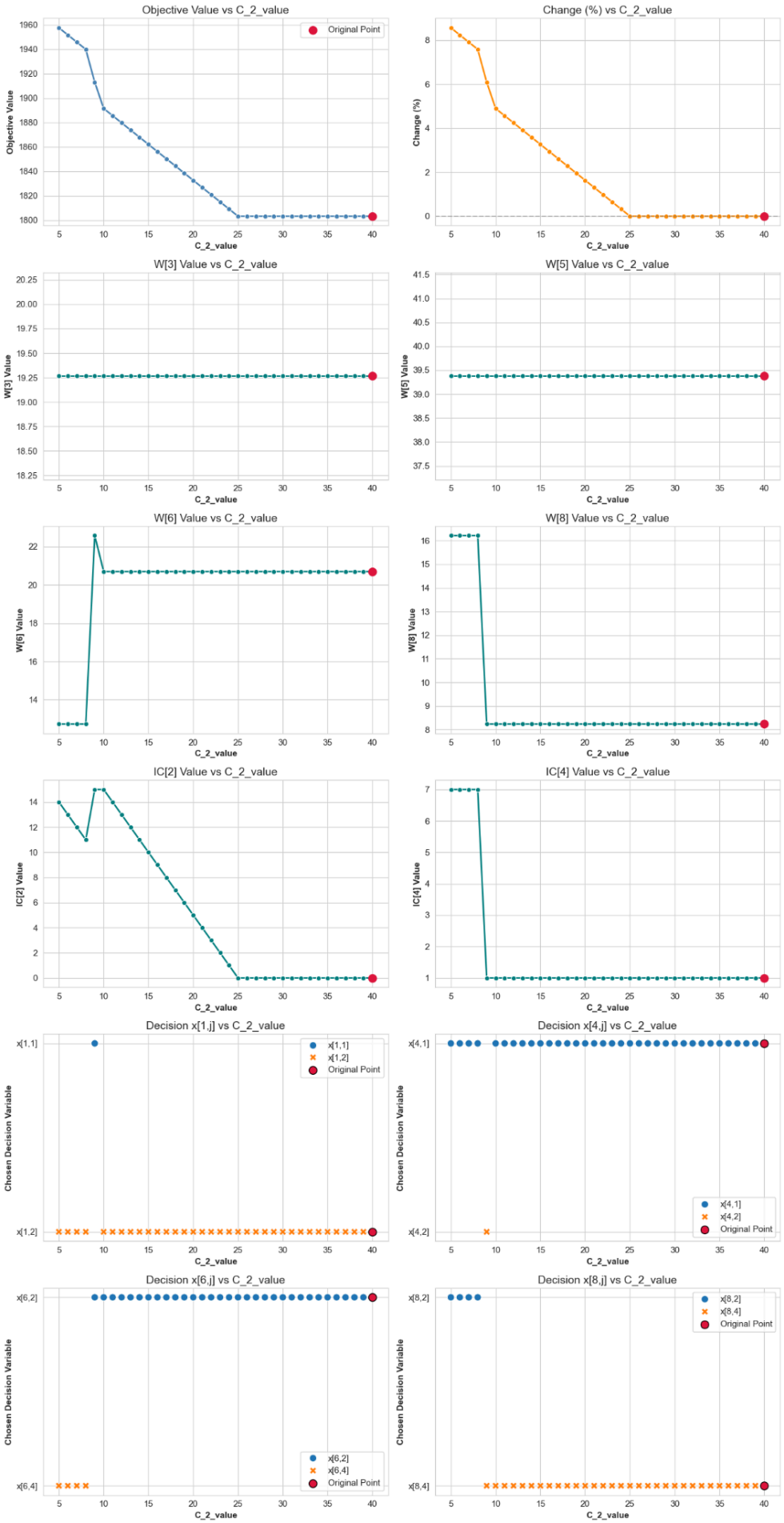
        plt.scatter(orig_x, df_final[var].iloc[orig_idx],
                    color=highlight_color, s=highlight_size,
                    label='Original Point', zorder=5)

        if var == 'pct_change':
            plt.axhline(0, color='gray', linestyle='--', alpha=0.5)

        plt.title(f'{ylabel} vs C_2_value')
        plt.xlabel('C_2_value')
        plt.ylabel(ylabel)

        if i == 1:
            plt.legend()

plt.tight_layout()
plt.show()
```


Objective Value vs C_2_value
Original Point
Objective Value
C_2_value
Change (%) vs C_2_value
Change (%)
C_2_value
W[3] Value vs C_2_value
W[3] Value
C_2_value
W[5] Value vs C_2_value
W[5] Value
C_2_value
W[6] Value vs C_2_value
W[6] Value
C_2_value
W[8] Value vs C_2_value
W[8] Value
C_2_value
IC[2] Value vs C_2_value
IC[2] Value
C_2_value
IC[4] Value vs C_2_value
IC[4] Value
C_2_value
Decision x[1,j] vs C_2_value
x[1,1]
x[1,2]
Original Point
Chosen Decision Variable
C_2_value
Decision x[4,j] vs C_2_value
x[4,1]
x[4,2]
Original Point
Chosen Decision Variable
C_2_value
Decision x[6,j] vs C_2_value
x[6,2]
x[6,4]
Original Point
Chosen Decision Variable
C_2_value
Decision x[8,j] vs C_2_value
x[8,2]
x[8,4]
Original Point
Chosen Decision Variable
C_2_value

مشاهده می‌شود با افزایش این پارامتر تابع هدف روند نزولی طی می‌کند و پس از مقدار ظرفیت 25 مقدار تابع هدف ثابت می‌ماند.
این نشان می‌دهد میتوان از تعداد ظرفیت کانتینر ها را تا 15 عدد کاهش دهیم و تغییری در هزینه ما ایجاد نمی‌شود.

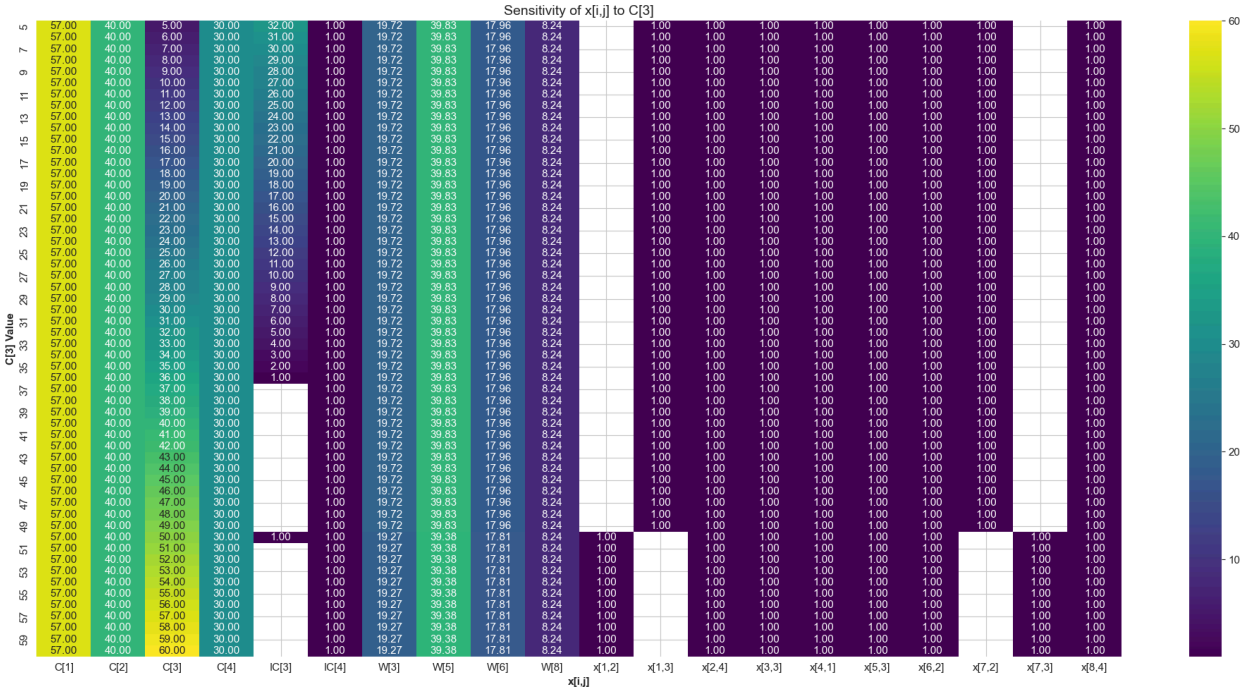
ظرفیت اسکله شماره 3)

همانند بخش قبل مقدار ظرفیت اسکله 3 را یکی یکی از 5 تا 40 افزایش داده اما به دلیل اینکه مقدار اولیه ظرفیت 52 بود و برای مقایسه دقیق تر ما ظرفیت اسکیه 3 را تا 60 هم افزایش داده‌ایم و حاصل مدل را در دیتا فریم ذخیره کردیم.
حال هیت مپ این بخش را مشاهده می‌کنیم:

```
# Convert wide-form df to long-form
df_long = df_final.melt(id_vars=['C_3_value', 'objective'],
var_name='(i,j)', value_name='x_val')

# Filter only x variables that changed meaningfully (optional)
df_long = df_long[df_long['x_val'].abs() > 1e-3]

# Heatmap of decision variable changes
pivot = df_long.pivot(index='C_3_value', columns='(i,j)', values='x_val')
plt.figure(figsize=(20, 10))
sns.heatmap(pivot, cmap="viridis", annot=True, fmt=".2f")
plt.title("Sensitivity of x[i,j] to C[3]")
plt.xlabel("x[i,j]")
plt.ylabel("C[3] Value")
plt.tight_layout()
plt.show()
```



در این بخش ابتدا تخصیص کشتی‌های 1 و 7 ایتدا به اسکله 3 بوده اما پس از افزایش ظرفیت اسکله 3 این دو کشتی به اسکله 2 تخصیص پیدا کرده‌اند.

برای بررسی تغییر این پارامتر در تابع هدف داریم:

```
# Style settings
sns.set_style("whitegrid")
plt.rcParams.update({
    'font.size': 11,
    'axes.labelweight': 'bold',
    'axes.titlesize': 14
})

# Colors & markers
main_color = 'steelblue'
highlight_color = 'crimson'
marker_style = 'o'
marker_size = 6
highlight_size = 100

# Original point index
orig_idx = 0
```

```
orig_x = df_final['C_3_value'].iloc[orig_idx]
orig_obj = df_final['objective'].iloc[orig_idx]

# Calculate percentage change
df_final['pct_change'] = ((df_final['objective'] - orig_obj) / orig_obj) *
100

# Base continuous plots
variables = [
    ('objective', 'Objective Value'),
    ('pct_change', 'Change (%)'),
]

# Add W[i] and IC[i] variables if present
for col in [ 'IC[3]']:
    if col in df_final.columns:
        variables.append((col, f'{col} Value'))

# Decision groups to check
decision_groups = {
    'Decision x[1,j]': ['x[1,1]', 'x[1,2]'],
    'Decision x[7,j]': ['x[7,2]', 'x[7,3]'],
}

# Filter valid decision groups
decision_plots = []
for label, cols in decision_groups.items():
    if all(col in df_final.columns for col in cols):
        decision_plots.append((cols, label))
        variables.append((label, 'Chosen Decision'))

# Layout
num_plots = len(variables)
cols = 2
rows = math.ceil(num_plots / cols)
```

```
# Plotting
plt.figure(figsize=(7 * cols, 4.5 * rows))

for i, (var, ylabel) in enumerate(variables, 1):
    plt.subplot(rows, cols, i)

    if var.startswith("Decision"):
        # Get correct group of columns
        for group_cols, label in decision_plots:
            if label == var:
                break

        # Prepare long-form DataFrame
        df_x = df_final[['C_3_value'] + group_cols].copy()
        df_x['Index'] = df_x.index
        df_long = df_x.melt(id_vars=['C_3_value', 'Index'],
                            value_vars=group_cols,
                            var_name='Decision', value_name='Chosen')
        df_long = df_long[df_long['Chosen'] == 1]

        # Plot switches
        sns.scatterplot(data=df_long, x='C_3_value', y='Decision',
                        hue='Decision', style='Decision', s=80)

        # Highlight original
        orig_dec = df_long[df_long['Index'] == orig_idx]
        plt.scatter(orig_dec['C_3_value'], orig_dec['Decision'],
                    color=highlight_color, s=highlight_size,
                    edgecolor='black', label='Original Point', zorder=5)

        plt.title(f'{label} vs C_3_value')
        plt.ylabel('Chosen Decision Variable')
        plt.xlabel('C_3_value')
        plt.legend()

    else:
```

```
        # Continuous line plots
        color = 'darkorange' if var == 'pct_change' else main_color
        if 'W[' in var or 'IC[' in var:
            color = 'teal'

        sns.lineplot(data=df_final, x='C_3_value', y=var,
                     marker=marker_style, color=color,
                     markersize=marker_size, linewidth=2)

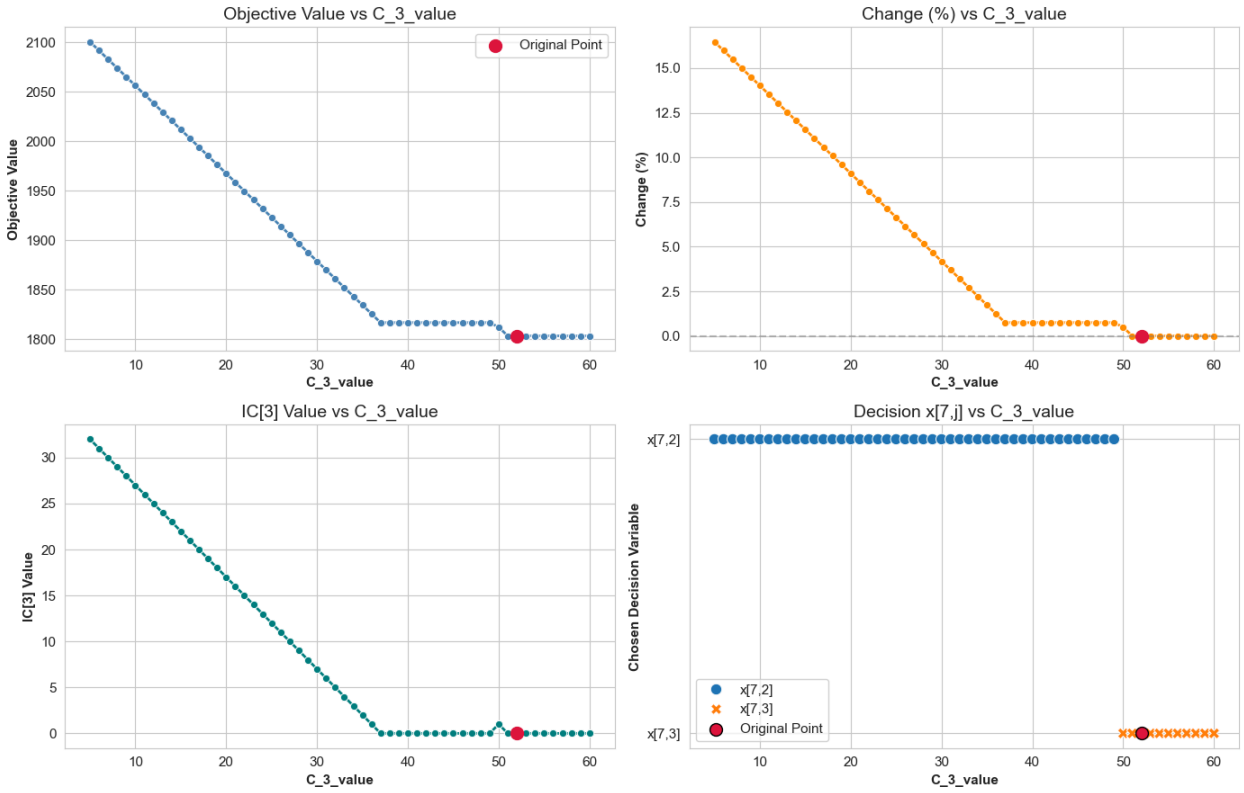
        plt.scatter(orig_x, df_final[var].iloc[orig_idx],
                    color=highlight_color, s=highlight_size,
                    label='Original Point', zorder=5)

        if var == 'pct_change':
            plt.axhline(0, color='gray', linestyle='--', alpha=0.5)

        plt.title(f'{ylabel} vs C_3_value')
        plt.xlabel('C_3_value')
        plt.ylabel(ylabel)

        if i == 1:
            plt.legend()

plt.tight_layout()
plt.show()
```



در این قسمت همراه با افزایش تعداد ظرفیت، مقدار تابع هدف کاهش می‌یابد سپس از مقدار 37 تا 49 مقدار تابع ثابت است، سپس یک کاهش کوچیکی باز رخ می‌دهد و سپس پس از مقدار 51 مقدار تابع هدف ثابت می‌ماند.

زمان رسیدن کشتی شماره 6)

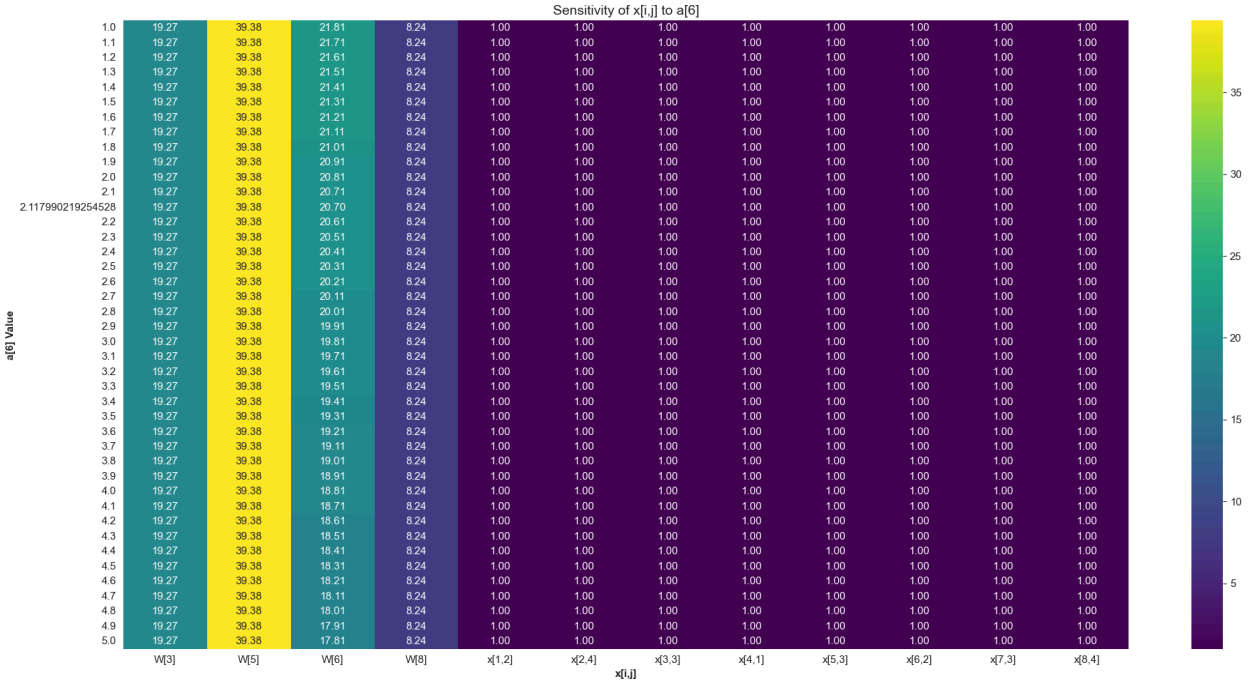
در این بخش همانند بخش‌های قبل میزان پارامتر از 1 تا 5 با گام‌های 0.1 تغییر داده و حل مدل را در آن ذخیره کرده.

```
plt.figure(figsize=(6, 4))

# Line plot with label
plt.plot(df_final['a_6_value'], df_final['objective'], marker='o',
label='Objective Curve', zorder=1)


plt.scatter(
    x=df_final['a_6_value'].iloc[0], y=df_final['objective'].iloc[0],
    color='red', s=100, linewidths=2,marker='*',
    label='Minimum Point', zorder=3
)
```

```
plt.xlabel('a[6] Value')
plt.ylabel('Objective Value')
plt.title('Objective Sensitivity to a[6]')
plt.grid(True, zorder=0)
plt.legend()
plt.tight_layout()
plt.show()
```



حال در هیت مپ بالا متوجه می‌شویم که زودتر یا دیرتر رسیدن کشتی 6 تغییری در متغیر های مسئله ایجاد نمی‌کند و فقط زمان انتظار خود کشتی 6 را کاهش می‌دهد.

همچنین در تابع هدف هم تغییر این پارامتر را بررسی کرده‌ایم:

```
# Style settings
sns.set_style("whitegrid")
plt.rcParams.update({
    'font.size': 11,
    'axes.labelweight': 'bold',
    'axes.titlesize': 14
})

# Colors & markers
main_color = 'steelblue'
```

```
highlight_color = 'crimson'
marker_style = 'o'
marker_size = 6
highlight_size = 100

# Original point index
orig_idx = 0
orig_x = df_final['a_6_value'].iloc[orig_idx]
orig_obj = df_final['objective'].iloc[orig_idx]

# Calculate percentage change
df_final['pct_change'] = ((df_final['objective'] - orig_obj) / orig_obj) *
100

# Base continuous plots
variables = [
    ('objective', 'Objective Value'),
    ('pct_change', 'Change (%)'),
]

# Add W[i] and IC[i] variables if present
for col in [ 'W[6]']:
    if col in df_final.columns:
        variables.append((col, f'{col} Value'))

# Decision groups to check
decision_groups = {
}

# Filter valid decision groups
decision_plots = []
for label, cols in decision_groups.items():
    if all(col in df_final.columns for col in cols):
        decision_plots.append((cols, label))
        variables.append((label, 'Chosen Decision'))
```

```
# Layout
num_plots = len(variables)
cols = 2
rows = math.ceil(num_plots / cols)

# Plotting
plt.figure(figsize=(7 * cols, 4.5 * rows))

for i, (var, ylabel) in enumerate(variables, 1):
    plt.subplot(rows, cols, i)

    if var.startswith("Decision"):
        # Get correct group of columns
        for group_cols, label in decision_plots:
            if label == var:
                break

        # Prepare long-form DataFrame
        df_x = df_final[['a_6_value'] + group_cols].copy()
        df_x['Index'] = df_x.index
        df_long = df_x.melt(id_vars=['a_6_value', 'Index'],
                            value_vars=group_cols,
                            var_name='Decision', value_name='Chosen')
        df_long = df_long[df_long['Chosen'] == 1]

        # Plot switches
        sns.scatterplot(data=df_long, x='a_6_value', y='Decision',
                        hue='Decision', style='Decision', s=80)

        # Highlight original
        orig_dec = df_long[df_long['Index'] == orig_idx]
        plt.scatter(orig_dec['a_6_value'], orig_dec['Decision'],
                    color=highlight_color, s=highlight_size,
                    edgecolor='black', label='Original Point', zorder=5)

        plt.title(f'{label} vs a_6_value')
```

```
        plt.ylabel('Chosen Decision Variable')
        plt.xlabel('a_6_value')
        plt.legend()

    else:
        # Continuous line plots
        color = 'darkorange' if var == 'pct_change' else main_color
        if 'W[' in var or 'IC[' in var:
            color = 'teal'

        sns.lineplot(data=df_final, x='a_6_value', y=var,
                     marker=marker_style, color=color,
                     markersize=marker_size, linewidth=2)

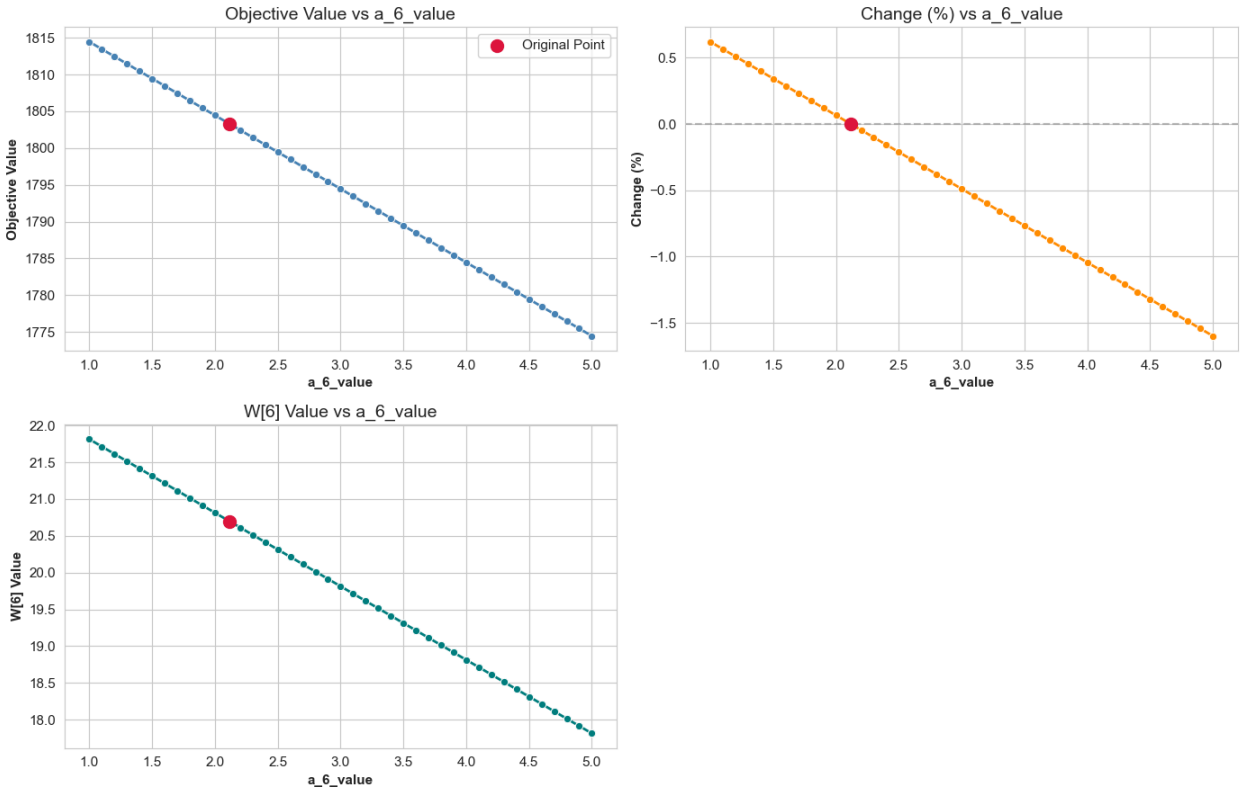
        plt.scatter(orig_x, df_final[var].iloc[orig_idx],
                    color=highlight_color, s=highlight_size,
                    label='Original Point', zorder=5)

        if var == 'pct_change':
            plt.axhline(0, color='gray', linestyle='--', alpha=0.5)

        plt.title(f'{ylabel} vs a_6_value')
        plt.xlabel('a_6_value')
        plt.ylabel(ylabel)

        if i == 1:
            plt.legend()

plt.tight_layout()
plt.show()
```



نمودار های بالا نشان می‌دهند که با دیرتر رسیدن کشتی 6 مقدار تابع هدف کاهش می‌یابد و به این دلیل است که کشتی 6 در صورت دیر رسیدن، زمان انتظار کمتر و در ادامه، هزینه کمتری را متحمل می‌شود.

گزارش بخش Gurobipy:

ما در این بخش با استفاده از کتابخانه Gurobipy، مسئله بهینه‌سازی تخصیص کشتی‌ها به اسکله‌ها و نحوه زمان‌بندی آن‌ها را حل نمودیم. در ابتدا، داده‌های مسئله مانند تعداد کشتی‌ها و اسکله‌ها، زمان‌های تخلیه، ظرفیت‌ها و زمان‌های ورود از یک فایل اکسل خواندیم. سپس، متغیر های تصمیم‌گیری مختلفی تعریف کردیم که شامل تخصیص کشتی به اسکله، زمان انتظار هر کشتی، و ترتیب سرویس‌دهی کشتی‌ها در یک اسکله می‌باشند. تابع هدف مدل را به گونه‌ای تعریف کردیم که مجموع هزینه‌های مربوط به زمان انتظار کشتی‌ها، زمان تخلیه و افزایش ظرفیت اسکله‌ها را کمینه کند. محدودیت‌های متعددی شامل تخصیص هر کشتی فقط به یک اسکله، رعایت ظرفیت و طول اسکله، ترتیب عملیات و حداکثر زمان انتظار را نیز برای مدل تعریف کردیم. در نهایت، مدل با استفاده از کتابخانه Gurobi حل شده و در صورتی که جواب بهینه پیدا شود، مقادیر متغیر ها و مقدار تابع هدف نمایش داده می‌شود.

در این بخش ابتدا کتابخانه‌های مربوطه را وارد کردیم:

```
import gurobipy as gp
from gurobipy import GRB
import pandas as pd
import ast
```

در این قسمت، مدل را تعریف و برای آن یک اسم تعیین کردیم:

```
model = gp.Model("OperationsResearchModel")
```

در مرحله بعد، ست‌ها را تعریف کردیم:

```
n = 8
m = 4
big_M = 10**8
ships_set = range(1, n + 1)
docks_set = range(1, m + 1)
iq_set = gp.tuplelist([(i, q) for i in ships_set for q in ships_set if i != q])
k_set = range(1, 3 + 1)
bin_set = range(1, 2 + 1)
```

پس از آن، پارامترها را از فایل‌های اکسل خواندیم:

```
Containers = pd.read_excel('Parameters.xlsx', sheet_name='Containers',
index_col=0)
Anchorage = pd.read_excel('Parameters.xlsx', sheet_name='Anchorage',
index_col=0)
Unload_Time_processing_df = pd.read_excel('Parameters.xlsx', sheet_name='Unload
Time')
Capacity_df = pd.read_excel('Parameters.xlsx', sheet_name='Capacity',
index_col=0)
Length_df = pd.read_excel('Parameters.xlsx', sheet_name='Length', index_col=0)
Max_length_df = pd.read_excel('Parameters.xlsx', sheet_name='Max Length',
index_col=0)
Lag_df = pd.read_excel('Parameters.xlsx', sheet_name='Lag', index_col=0)
Priority_df = pd.read_excel('Parameters.xlsx', sheet_name='Priority',
index_col=0)
Arrival_Time_df = pd.read_excel('Parameters.xlsx', sheet_name='Arrival Time',
index_col=0)
Cost_for_Extra_Capacity_df = pd.read_excel('Parameters.xlsx', sheet_name='Cost
for Extra Capacity', index_col=0)
```

```
Max_Extra_Allowable_Capacity_df = pd.read_excel('Parameters.xlsx',
sheet_name='Max Extra Allowable Capacity', index_col=0)
```

در این قسمت، پارامترها را به دیکشنری تبدیل کردیم:

```
Unload_Time_processing_df['Ship_Berth'] =
Unload_Time_processing_df['Ship_Berth'].apply(lambda x: tuple(map(int,
ast.literal_eval(x))))
unload_dict = dict(zip(Unload_Time_processing_df['Ship_Berth'],
Unload_Time_processing_df['Unload_Time']))
C_param = Capacity_df['Capacity'].to_dict()
CA_param = Containers['Containers'].to_dict()
L_param = Max_length_df['Max_Length'].to_dict()
LS_param = Length_df['Length'].to_dict()
P_param = Priority_df['Priority'].to_dict()
O_param = Lag_df['Lag'].to_dict()
a_param = Arrival_Time_df['Arrival_Time'].to_dict()
CIC_param = Cost_for_Extra_Capacity_df['Cost for Extra Capacity'].to_dict()
ext_param = Max_Extra_Allowable_Capacity_df['Max Extra Allowable
Capacity'].to_dict()
```

پس از آن، متغیرها را تعریف نمودیم:

```
X = model.addVars(ships_set, docks_set, vtype=GRB.BINARY, name="X")
W = model.addVars(ships_set, vtype=GRB.CONTINUOUS, lb=0.0, name="W")
IC = model.addVars(docks_set, vtype=GRB.INTEGER, lb=0, name="IC")
Y = model.addVars(iq_set, docks_set, vtype=GRB.BINARY, name="y") # Using iq_set
defined earlier
U = model.addVars(ships_set, k_set, vtype=GRB.CONTINUOUS, lb=0.0, name="u")
F = model.addVars(ships_set, bin_set, vtype=GRB.BINARY, name="f")
START = model.addVars(ships_set, vtype=GRB.CONTINUOUS, lb=0.0, name="start")
```

بعد از تعریف متغیرها، تابع هدف را ساختیم:

```
obj_expr = (
    25 * gp.quicksum(U[i, 1] for i in ships_set) +
    10 * gp.quicksum(U[i, 2] for i in ships_set) +
    5 * gp.quicksum(U[i, 3] for i in ships_set) +
    5 * gp.quicksum(unload_dict[(i, j)] * X[i, j] for i in ships_set for j in
docks_set if (i,j) in unload_dict) +
    gp.quicksum(CIC_param[j] * IC[j] for j in docks_set)
)
model.setObjective(obj_expr, GRB.MINIMIZE)
```

در این قسمت، محدودیت‌های مختلف را به مسئله اضافه کردیم:

```
model.addConstrs((gp.quicksum(X[i, j] for j in docks_set) == 1
                  for i in ships_set), name="c1_one_pier_to_ship")

for j in docks_set:
    if j == 3:
        model.addConstr(gp.quicksum(X[i, j] for i in ships_set) == 3,
name=f"c2_ships_at_pier_{j}")
    else:
        model.addConstr(gp.quicksum(X[i, j] for i in ships_set) <= 2,
name=f"c2_ships_at_pier_{j}")

model.addConstrs((gp.quicksum(X[i, j] * CA_param[i] for i in ships_set) <=
C_param[j] + IC[j]
                  for j in docks_set), name="c3_container_capacity")

model.addConstrs((IC[j] <= ext_param[j]
                  for j in docks_set), name="c4_extra_capacity_ceiling")

model.addConstrs((LS_param[i] * X[i, j] <= L_param[j]
                  for i in ships_set for j in docks_set),
name="c5_ship_length")

model.addConstrs((W[i] <= 80 * (1 - P_param[i])
                  for i in ships_set), name="c6_max_wait")

model.addConstrs((W[i] == START[i] - a_param[i]
                  for i in ships_set), name="c7_wait_time_def")

for i_ship, q_ship in iq_set:
    for j_dock in docks_set:
        if (i_ship, j_dock) in unload_dict:
             model.addConstr(
                 a_param[i_ship] + W[i_ship] + unload_dict[(i_ship, j_dock)] +
O_param[j_dock] <=
                 START[q_ship] + big_M * (1 - Y[i_ship, q_ship, j_dock]),
                 name=f"c8_not_unloading_{i_ship}_{q_ship}_{j_dock}"
             )

for i_ship, q_ship in iq_set:
    for j_dock in docks_set:
        model.addConstr(Y[i_ship, q_ship, j_dock] + Y[q_ship, i_ship, j_dock]
```

```
<= 1,
                        name=f"c9_y_sum_le_1_{i_ship}_{q_ship}_{j_dock}")
        model.addConstr(Y[i_ship, q_ship, j_dock] + Y[q_ship, i_ship, j_dock]
>=
                        X[i_ship, j_dock] + X[q_ship, j_dock] - 1,
                        name=f"c10_y_sum_ge_X_{i_ship}_{q_ship}_{j_dock}")
        model.addConstr(Y[i_ship, q_ship, j_dock] + Y[q_ship, i_ship, j_dock]
<= X[i_ship, j_dock],
                        name=f"c11_y_sum_le_Xi_{i_ship}_{q_ship}_{j_dock}")

model.addConstrs((START[i] >= a_param[i]
                  for i in ships_set), name="c12_start_after_arrival")

for i_ship in ships_set:
    j_dock = 3
    if j_dock in docks_set:
        sum_y_ijk = gp.quicksum(Y[i_ship, k_ship, j_dock] for k_ship in
ships_set if k_ship != i_ship)
        model.addConstr(W[i_ship] <= 80 * (2 - sum_y_ijk),
                        name=f"c13_wait_zero_dock3_{i_ship}")

for i_ship, q_ship in iq_set:
    for j_dock in docks_set:
        if j_dock != 3:
            model.addConstr(W[i_ship] <= 80 * (1 - Y[i_ship, q_ship, j_dock]),

name=f"c13_wait_zero_otherdocks_{i_ship}_{q_ship}_{j_dock}")

model.addConstrs((W[i] == gp.quicksum(U[i, k] for k in k_set)
                  for i in ships_set), name="c14_W_equals_sum_U")

for i in ships_set:
    for b_idx in bin_set:
        if b_idx == 1:
            model.addConstr(10 * F[i, b_idx] <= U[i, b_idx],
name=f"c15_F_U_rel_{i}_{b_idx}")
        elif b_idx == 2:
            model.addConstr(30 * F[i, b_idx] <= U[i, b_idx],
name=f"c15_F_U_rel_{i}_{b_idx}")

for i in ships_set:
    for k_idx in k_set:
        if k_idx == 1:
            model.addConstr(U[i, k_idx] <= 10, name=f"c16_U_bound_{i}_{k_idx}")
```

```
        elif k_idx == 2:
            model.addConstr(U[i, k_idx] <= 30 * F[i, 1],
name=f"c16_U_bound_{i}_{k_idx}")
        elif k_idx == 3:
            model.addConstr(U[i, k_idx] <= 30 * F[i, 2],
name=f"c16_U_bound_{i}_{k_idx}")

model.addConstrs((F[i, 1] >= F[i, 2]
                  for i in ships_set), name="c17_F_ordering")
```

پس از اضافه کردن محدودیت‌ها، مسئله را حل کردیم:

```
model.optimize()
```

در انتها هم جواب بهینه را درصورت وجود، نشان دادیم:

```
if model.Status == GRB.OPTIMAL:
    print("\nOptimal solution found!")
    print(f"Objective value: {model.ObjVal:.4f}")

    print("\n--- Variable Values ---")
    for i in ships_set:
        for j in docks_set:
            if X[i, j].X > 0.5:
                print(f"X[{i}, {j}] = {X[i, j].X:.0f} (Ship {i} to Dock {j})")

    print("\nWaiting Times (W):")
    for i in ships_set:
        if W[i].X > 1e-6:
            print(f"W[{i}] = {W[i].X:.4f}")

    print("\nStart Times (start):")
    for i in ships_set:
        if START[i].X > 1e-6:
            print(f"start[{i}] = {START[i].X:.4f}")

    print("\nIncreased Capacities (IC):")
    for j in docks_set:
        if IC[j].X > 0.5:
             print(f"IC[{j}] = {IC[j].X:.0f}")

    print("\nPrecedence Variables (y):")
    for i_ship, q_ship in iq_set:
```

```
        for j_dock in docks_set:
            if Y[i_ship, q_ship, j_dock].X > 0.5:
                print(f"y[{i_ship},{q_ship},{j_dock}] = {Y[i_ship, q_ship,
j_dock].X:.0f} (Ship {i_ship} precedes {q_ship} at Dock {j_dock})")

    print("\nWaiting Time Components (u):")
    for i in ships_set:
        for k_idx in k_set:
            if U[i, k_idx].X > 1e-6:
                print(f"u[{i},{k_idx}] = {U[i, k_idx].X:.4f}")

    print("\nBinary Indicators for U (f):")
    for i in ships_set:
        for b_idx in bin_set:
            if F[i, b_idx].X > 0.5:
                print(f"f[{i},{b_idx}] = {F[i, b_idx].X:.0f}")

elif model.Status == GRB.INFEASIBLE:
    print("\nModel is infeasible.")
    print("Computing IIS to help identify conflicting constraints...")
    model.computeIIS()
    model.write("model_iis.ilp")
    print("IIS written to model_iis.ilp")
elif model.Status == GRB.UNBOUNDED:
    print("\nModel is unbounded.")
else:
    print(f"\nOptimization ended with status: {model.Status}")
```